



CS329Z: Engineering AI Agents

Frameworks & Orchestration

Diyi Yang, Michael Ryan, John Yang

Announcements

- Project Proposal is due **Friday 10/9 at 11:59pm** on Gradescope!
 - Submit as a group, please do not submit the same proposal individually!
- Apply for Google Credits by this Friday! Enrolled students should have received an email from John.
 - You should already have access to OpenRouter credits courtesy of Laude!
- Homework 1 was released, due on **Friday October 30th at 11:59pm**.
 - Start early! It's a new homework and it's pretty long!



Quick Poll

Who in this room has used

 PyTorch?



Abstractions in Frameworks

Writing Backprop by hand

```
# Forward pass
pred = w * x + b
loss = np.mean((pred - y) ** 2)

# Backpropagation
d_pred = 2 * (pred - y) / y.size
d_w, d_b = np.sum(d_pred * x), np.sum(d_pred)

# Update
w, b = w - lr * d_w, b - lr * d_b
```

Writing Backprop with PyTorch

```
optimizer = torch.optim.SGD([w, b], lr=0.1)
optimizer.zero_grad()

# Forward pass
pred = w * x + b
loss = ((pred - y) ** 2).mean()

# Backpropagation
loss.backward()

# Update
optimizer.step()
```



Frameworks abstract away some of the underlying system details.

In the rest of the course you must implement/understand the **system details**.

The goal for today is to learn the **abstractions**.



Learning Objectives

- **Identify** common frameworks used in agentic system engineering.
- **Relate** the abstractions introduced by these frameworks back to the underlying system designs we have discussed in the course.
- **Assess** the usefulness of different tools for agent orchestration and reliability.



Lecture Overview

- What does it mean to “Engineer” Agents? (10 mins)
- Frameworks for Workflows and Programs (25 mins)
- Open Source Agent Harnesses (15 mins)
- Orchestration and Telemetry (15 mins)
- Best Practices and Challenges (15 mins)



Want to follow along with code today?



```
git clone https://github.com/cs329z/lecture5-frameworks.git  
cd lecture5-frameworks  
./setup.sh
```

This lecture notebook will use about \$1 or less of OpenRouter credits.



What does it mean to “Engineer” Agents?

“Engineer” Agents?

Workflows

Agent Harnesses

Orchestration

Best Practices



Three Pillars of AI Systems

Data

- Training Data
- Rollouts
- Demonstrations
- Human Feedback
- ...

System or Model

- Language Model
- Compound AI System / Workflow
- Agentic System
- ...

Evaluation + Metric

- Pass Rate
- Exact Match
- LLM Judge
- Cost/Latency
- ...



Three Pillars of AI Systems

Data

- Training Data
- Rollouts
- Demonstrations
- Human Feedback
- ...

System or Model

- Language Model
- Compound AI System / Workflow
- Agentic System
- ...

Evaluation + Metric

- Pass Rate
- Exact Match
- LLM Judge
- Cost/Latency
- ...

AI System



Three Pillars of AI Systems

Data

- Training Data
- Rollouts
- Demonstrations
- Human Feedback
- ...

System or Model

- Language Model
- Compound AI System / Workflow
- Agentic System
- ...

Evaluation + Metric

- Pass Rate
- Exact Match
- LLM Judge
- Cost/Latency
- ...



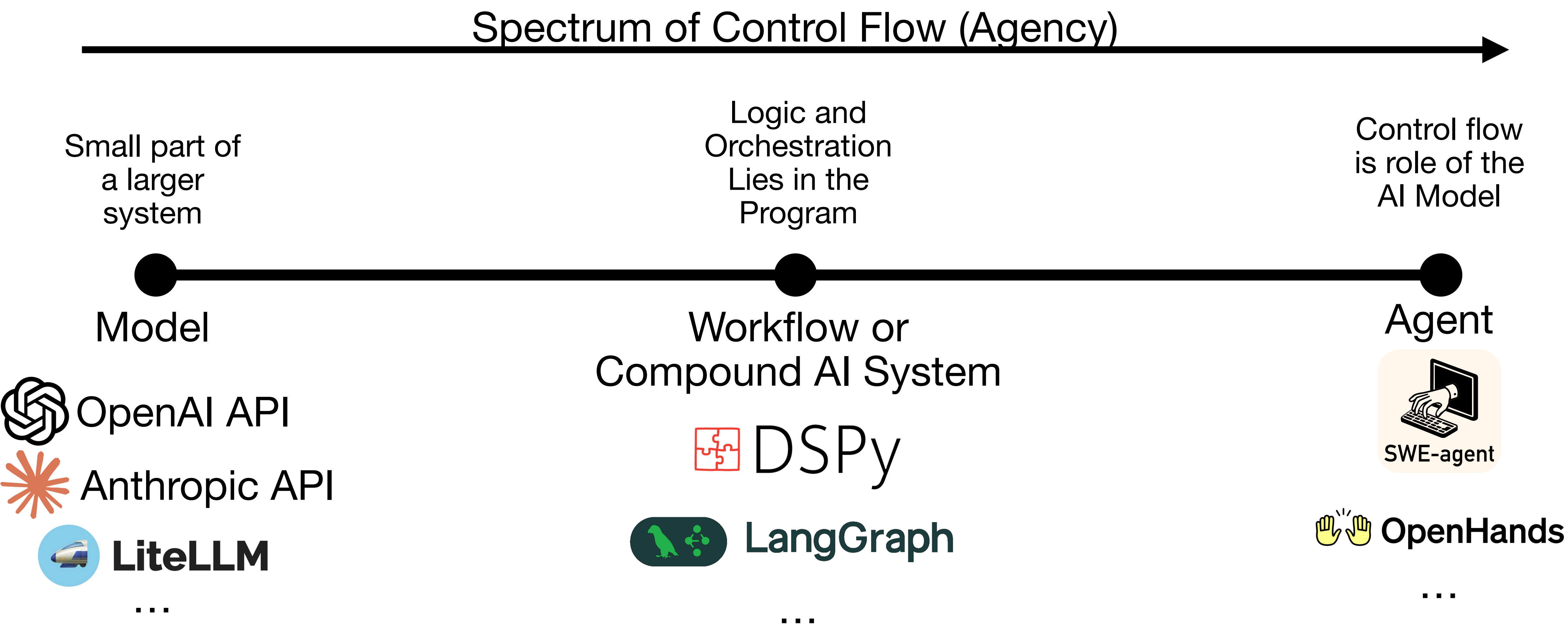
This Course

Introduction – What Are Agentic Systems?	LLMs for Builders
Retrieval-Augmented Generation (RAG)	Tool Use & Function Calling
Frameworks & Orchestration	Agent Design Patterns & Scaffolds
Agent Memory Architectures	Multi-Agent Systems
Optimization	What Data Do Agents Need?
Data Selection & Quality	Evaluation Fundamentals & Benchmark Design
LLM-as-Judge & Evaluation Infrastructure	Agent Safety & Guardrails
Coding & Software Agents	Proactive Agents
Frontiers & Open Problems	

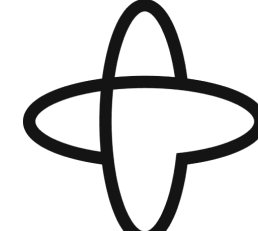
	System or Model
	Data
	Evaluation + Metric
	Emerging Topics

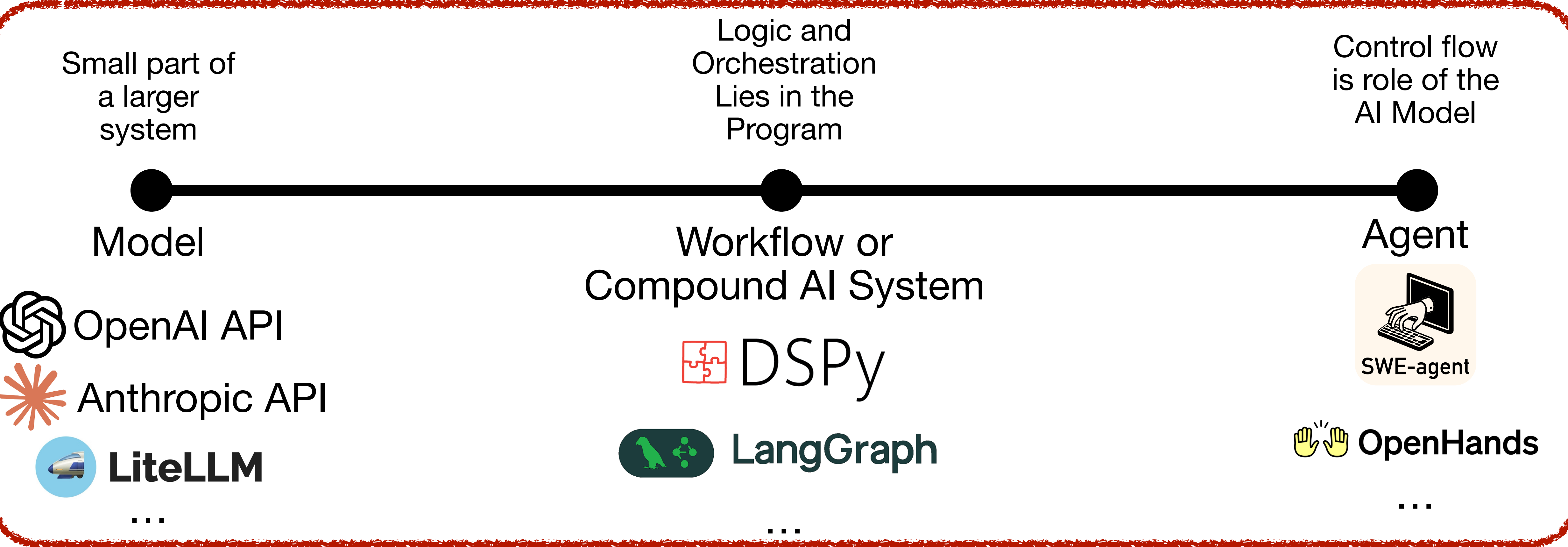


Framework Landscape



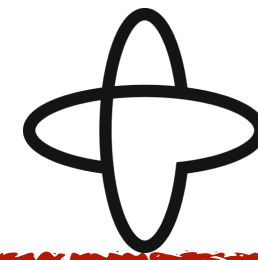
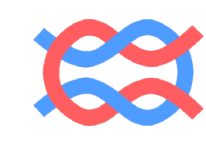
Framework Landscape

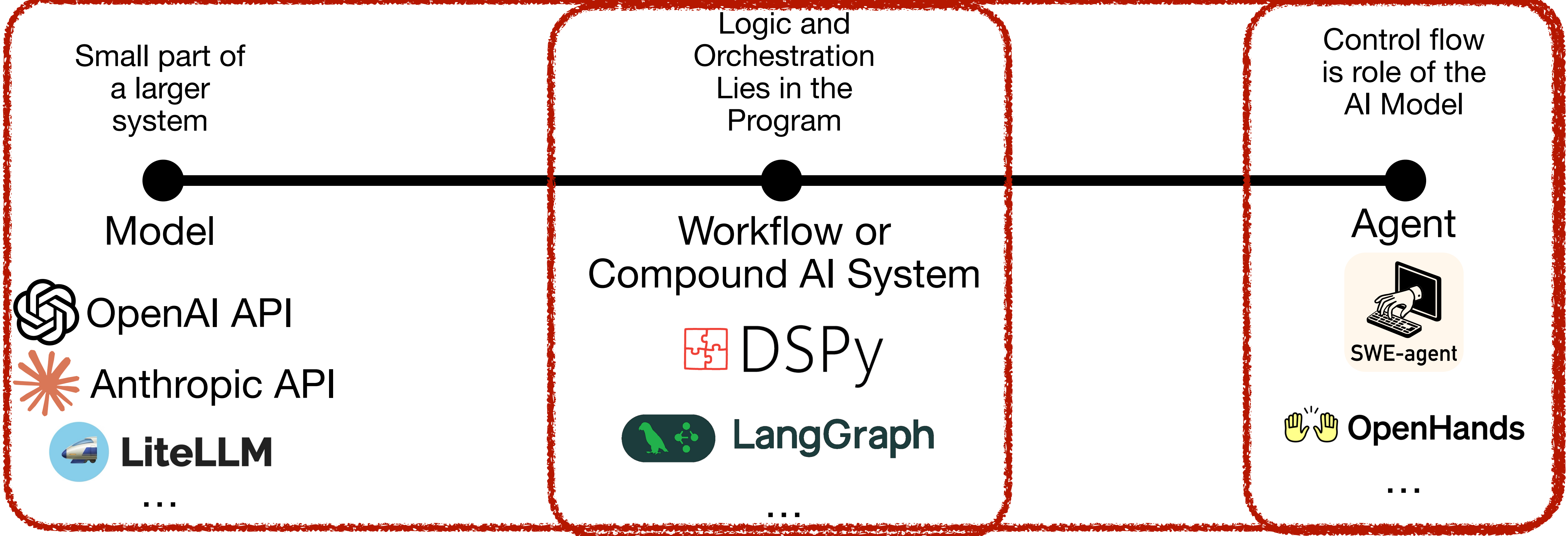
Reliability:  Temporal, ... **Observability:**  Langfuse, *mlflow*, ...



Framework Landscape

Part 3

Reliability:  Temporal, ... **Observability:**  Langfuse, *mlflow*, ...



Part 1

Part 2

Are these frameworks actually used?

 **dspy** Public ★ Starred 38.5k

 **langgraph** Public ★ Starred 42.8k

 **temporal** Public ★ Starred 23.5k

<https://agentic-engineering-jobs.com/jobs/dspy/sa...>

DSPy Developer Jobs in San Francisco [Visit >](#)

Build AI agents and RAG pipelines with DSPy at companies hiring in San Francisco. across 1,135 listings. a job →(1136 active listings)
Turing \$260K - \$320K/ ...

[Share](#) [Save](#) [Remove result](#) [Feedback](#)

 **Indeed**
<https://www.indeed.com/q-langgraph-jobs>

Now Hiring: 2000 Langgraph Jobs

Browse 2610 Langgraph jobs. New jobs posted today. build multi-agent conversational systems leveraging LangGraph, AutoGen, CrewAI, ADK, or similar frameworks.
Missing: listings | Show results with: [listings](#)



Recap

- Engineering AI Systems requires three core components: Data, System, Evals.
- Agency spectrum for ai systems ranges from models ➡ workflows ➡ agents.
- There are frameworks for each level of the spectrum.
- Some frameworks are dedicated to improving the reliability and observability of agents.



Frameworks for Workflows and Programs

“Engineer” Agents?

Workflows

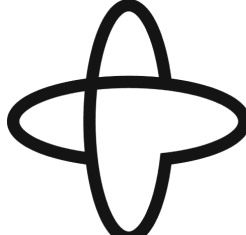
Agent Harnesses

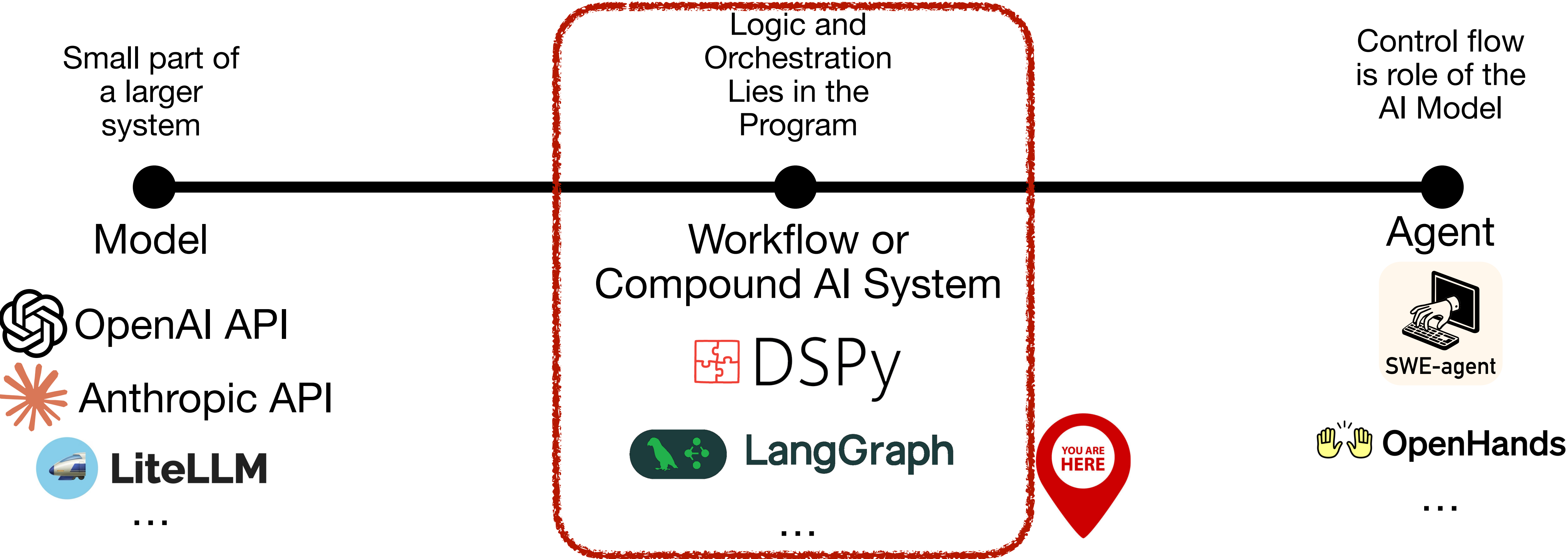
Orchestration

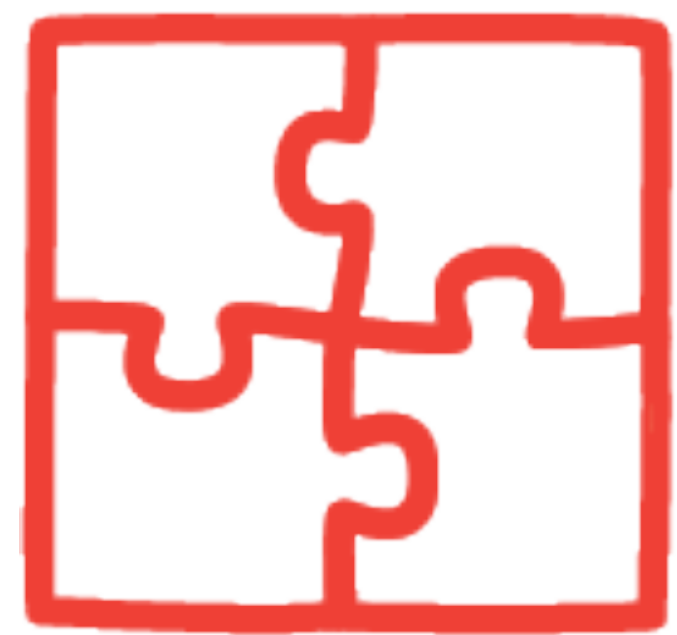
Best Practices



Framework Landscape

Reliability:  Temporal, ... **Observability:**  Langfuse, mlflow, ...





DSPy



Signatures *(What)*

Strictly define Inputs/Outputs to the Language Model.

Modules *(How)*

Compose signatures and program logic into callable functions.

Optimizers

Improve the prompts or weights of the program to maximize metrics.



Live Programming Demo



Signatures



Signatures

```
1 # The simplest signatures are string signatures
2
3 signature = "message -> response"
4 chat = dsp.Predict(signature)
5
6 print(chat(message="Hello, how are you?").response)
```

Hello! I'm doing well—thanks for asking. How are you today?



Signatures

```
1 # Signatures are great for structured input and output
2
3 sentiment_signature = "message -> sentiment"
4 sentiment_classifier = dsp.Predict(sentiment_signature)
5
6 print(sentiment_classifier(message="I am very happy today!").sentiment)
7 print(sentiment_classifier(message="I am very angry today!").sentiment)
8 print(sentiment_classifier(message="I am very confused today!").sentiment)
```

Python

positive

Negative

negative

Not good that these are not following same format...



Signatures

```
1 # We can apply types
2 typed_signature = "message: str -> sentiment: " \
3     "Literal['positive','negative','neutral'], confidence: float"
4 typed_sentiment_classifier = dsp.Predict(typed_signature)
5
6 print(typed_sentiment_classifier(message="I am very happy today!"))
7 print(typed_sentiment_classifier(message="I am very angry today!"))
8 print(typed_sentiment_classifier(message="I am very confused today!"))
```

✓ 0.1s

```
Prediction(
    sentiment='positive',
    confidence=0.98
)
Prediction(
    sentiment='negative',
    confidence=0.9
)
Prediction(
    sentiment='negative',
    confidence=0.86
)
```



Signatures

```
1 # When your signature gets more complicated
2 #     you may want to use a Signature class
3 import datetime as dt
4
5 class ExtractEvent(dspy.Signature):
6     """Extract event details from an email."""
7     email: str = dspy.InputField()
8     event_name: str | None = dspy.OutputField()
9     date: dt.date | None = dspy.OutputField()
10    start: dt.time | None = dspy.OutputField(desc="24-hour clock")
11    duration_minutes: int | None = dspy.OutputField(desc="duration of the event")
12
13    extract = dspy.Predict(ExtractEvent)
14    extract(email="We have a meeting scheduled for this Friday," \
15            " October 9th at 10 AM PST.")
```

✓ 0.8s

Python

```
Prediction(
    event_name='meeting',
    date=datetime.date(2026, 10, 9),
    start=datetime.time(10, 0),
    duration_minutes=None
)
```



Modules



Modules

```
1 # Sometimes you may want a different inference
2 #     strategy than the default "Predict"
3
4 extract_cot = dspy.ChainOfThought(ExtractEvent)
5 extract_cot(email="We have a meeting scheduled for this Friday," \
6               " October 9th at 10 AM PST.")
```

Python

```
Prediction(
  reasoning='The email states there is a meeting "this Friday, October 9th at 10 AM PST." So the event name is "
  event_name='meeting',
  date=datetime.date(2026, 10, 9),
  start=datetime.time(10, 0),
  duration_minutes=None
)
```



Modules

```
1 # A second, tiny LLM call we can compose with the first one
2
3 class PickEmoji(dspy.Signature):
4     """Pick one emoji that best represents a calendar event."""
5     event_name: str = dspy.InputField()
6     emoji: str = dspy.OutputField(desc="exactly one emoji character")
7
8 pick_emoji = dspy.ChainOfThought(PickEmoji)
9 pick_emoji(event_name="Dentist")
```

✓ 0.8s

Python

```
Prediction(
    reasoning='A dentist appointment is a healthcare-related calendar event, so a dental-themed emoji fits best.',
    emoji='🦷'
)
```



Modules

```
1 # Modules are composable and can include python logic
2
3 class UpdateCalendarFromEmail(dspy.Module):
4     def __init__(self):
5         self.extract = dspy.ChainOfThought(ExtractEvent) # LLM call 1
6         self.pick_emoji = dspy.ChainOfThought(PickEmoji) # LLM call 2
7
8     def forward(self, email: str):
9         details = self.extract(email=email)
10
11         # Plain Python control flow: not every email contains an event.
12         if details.event_name is None or details.date is None or details.start is None:
13             return dspy.Prediction(event=None, status="no event found in email")
14
15         emoji = self.pick_emoji(event_name=details.event_name).emoji
16         event = tinycal.add_event(
17             title=details.event_name,
18             date=details.date,
19             start=details.start,
20             duration_minutes=details.duration_minutes, # tinycal assumes 1 hour if None
21             emoji=emoji,
22         )
23         return dspy.Prediction(event=event, status="added to calendar")
```



Modules

```
1 update_calendar = UpdateCalendarFromEmail()  
2 print(update_calendar(email="We have a meeting scheduled for this Friday, October 9th at 10 AM PST."))  
3 print(update_calendar(email="Thanks for sending the slides, they look great!"))
```

✓ 2.2s

Python

```
Prediction(  
    event={'id': 'c5d03fe2', 'notes': '', 'emoji': '📅', 'title': 'meeting', 'date': '2026-10-09', 'start': '10:00',  
    status='added to calendar'  
)  
Prediction(  
    event=None,  
    status='no event found in email'  
)
```



Modules

tinycal

<

Today

>

Oct 4 – Oct 10, 2026

live

Clear

Sun4 no events	Mon5 no events	Tue6 no events	Wed7 no events	Thu8 no events	Fri9 10:00am – 11:00am 17 Meeting	Sat10 no events
--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--	---------------------------------------

Optimizers



Optimizers

Data

- Training Data
- Rollouts
- Demonstrations
- Human Feedback
- ...

System or Model

- Language Model
- Compound AI System / Workflow
- Agentic System
- ...

Evaluation + Metric

- Pass Rate
- Exact Match
- LLM Judge
- Cost/Latency
- ...



Optimizers: The Data

```
1 # 1. The data: 50 realistic emails with gold labels (see data/emails.json)
2 import json, random
3
4 rows = json.load(open("data/emails.json"))
5 print(rows[0]["email"])
6 print("GOLD:", {k: v for k, v in rows[0].items() if k != "email"})
```

✓ 0.0s

Appointment reminder: haircut with Jesse on Wednesday, October 28, 2026 at 2:45pm. It will run for one hour.

The Cut Shop

GOLD: {'event_name': 'haircut', 'date': '2026-10-28', 'start': '14:45', 'duration_minutes': 60}



Optimizers: The Data

```
1 # Turn them into dspy.Examples. Input: email. Labels: everything else.
2 examples = [dspy.Example(**row).with_inputs("email") for row in rows]
3 random.Random(0).shuffle(examples)
4 trainset, testset = examples[:30], examples[30:]
5 print(len(trainset), "train /", len(testset), "test")
```

✓ 0.0s

30 train / 20 test



Optimizers: The Metric

```
1 # 2. The metric: 1 if the date, start time, and duration ALL exactly match the gold label, else 0.
2 #   (event_name is free-form text, so we don't grade it.)
3 #   It also returns feedback text: GEPA shows this to the reflection model.
4 def event_metric(gold, pred, trace=None, pred_name=None, pred_trace=None):
5     checks = {
6         "date": str(pred.date) == gold.date,
7         "start": str(pred.start)[:5] == gold.start,
8         "duration_minutes": pred.duration_minutes == gold.duration_minutes,
9     }
10    score = float(all(checks.values()))
11    wrong = [f"{k}: expected {getattr(gold, k)!r}, got {getattr(pred, k)!r}" for k, ok in checks.items()
12            if not ok]
13    feedback = "All fields correct." if not wrong else "Wrong -> " + "; ".join(wrong)
14    return dspy.Prediction(score=score, feedback=feedback)
15 evaluate = dspy.Evaluate(devset=testset, metric=event_metric, num_threads=8, display_progress=True)
```

✓ 0.0s



Optimizers

```
1 # Sanity check: our big model (gpt-5.4-nano) should ace this
2 extract_big = dspy.Predict(ExtractEvent) # uses dspy_llm (gpt-5.4-nano), the default LM
3 evaluate(extract_big)
```

✓ 0.7s

Average Metric: 20.00 $\angle$ 20 (100.0%): 100%|██████████| 20/20 [00:00<00:00, 30.63it/s]

2026/10/03 22:30:20 INFO dspy.evaluate.evaluate: Average Metric: 20.0 $\angle$ 20 (100.0%)

EvaluationResult(score=100.0, results=<list of 20 results>)



Optimizers

```
1 # 3. The small model: Gemma 3 12B.
2 small_lm = dspy.LM("openrouter/google/gemma-3-12b-it", api_key=OPENROUTER_API_KEY, max_tokens=1024, temperature=0.0)
3
4 extract_small = dspy.Predict(ExtractEvent) # same signature...
5 extract_small.set_lm(small_lm)           # ...but this predictor uses the small model
6
7 # 4. Score the small model BEFORE optimization
8 before = evaluate(extract_small)
```

✓ 0.1s

Python

Average Metric: 11.00 / 20 (55.0%): 100%|██████████| 20/20 [00:00<00:00, 328.14it/s]

2026/10/03 22:45:14 INFO dspy.evaluate.evaluate: Average Metric: 11.0 / 20 (55.0%)



Optimizers

```
1 # 5. Optimize with GEPA on a small budget (100 metric calls, ~3 minutes).
2 #   The big model (dspy_llm) reflects on the small model's mistakes; the small model is what gets tuned.
3 gepa = dspy.GEPA(metric=event_metric, max_metric_calls=100, reflection_lm=dspy_llm, num_threads=8)
4 extract_optimized = gepa.compile(extract_small, trainset=trainset)
```

✓ 1.5s



Optimizers

```
1 # 6. Score the small model AFTER optimization
2 after = evaluate(extract_optimized)
3 print(f"small model before: {before.score:.1f}%    after: {after.score:.1f}%")
```

✓ 0.2s

Average Metric: 20.00 $\angle$ 20 (100.0%): 100%|██████████| 20/20 [00:00<00:00, 103.08it/s]

2026/10/03 22:30:22 INFO dspy.evaluate.evaluate: Average Metric: 20.0 $\angle$ 20 (100.0%)

small model before: 55.0% after: 100.0%



Optimizers

```
1 # The only thing GEPA changed is the instructions text:
2 print(extract_optimized.signature.instructions)
```

✓ 0.0s

Python

You are given an email (plain text). Your job is to extract a single calendar event from the email and output structured

Input format:

- A single message body as text (may include greetings, signatures, multiple sentences, and sometimes an “appointment reminder”)
- The event details may appear as:
 - Event name/description (e.g., “Phone Interview”, “Q3 budget review”, “haircut with Jesse”)
 - Date in natural language (e.g., “5 November 2026”, “Nov 3, 2026”, “Wednesday, October 28, 2026”)
 - Start time in natural language (e.g., “10:00 AM”, “11:00”, “2:45pm”, sometimes without timezone)
 - Duration (e.g., “45 minutes”, “about 30 minutes”, “run for one hour”)
 - Sometimes timezone is not mentioned; if absent, use UTC (+00:00) for outputs that include a timezone.

Output format (always produce these fields):

- event_name: short event title/summary extracted from the email (prefer the main activity/meeting type, not the sender name)
- date: ISO date in YYYY-MM-DD
- start: ISO time in HH:MM:SS with timezone if available/assumed:
 - If no timezone is mentioned, output start as HH:MM:SS+00:00 (i.e., UTC)
 - If the email includes a timezone, reflect it if present in the original text; otherwise follow the UTC rule
- duration_minutes: integer number of minutes if the email specifies it; otherwise output null/None (but only when truly applicable)

Extraction rules and pitfalls (must follow):



Agents



Agents

```
1 # An agent in one line. Specify the inputs/outputs and the tools it can use.
2 agent = dsp.ReAct("request -> summary", tools=tinycal.TOOLS)
3
4 result = agent(request="Schedule a 30-minute coffee with John on Oct 7, 2026 in the afternoon after my lectures, "
5 | | | | | "and put a book emoji on every one of my lectures for that same week.")
6 print(result.summary)
```

✓ 8.3s

- Added 📖 to lectures:
 - CS329Z Lecture – Oct 5, 2026, 13:30–14:50
 - CS329Z Lecture – Oct 7, 2026, 13:30–14:50
- Scheduled:
 - Coffee with John – Oct 7, 2026, 15:00–15:30 (☕)



Agents

tinycal

<

Today

>

Oct 4 – Oct 10, 2026

live

Clear

Sun4 no events	Mon5 1:30pm – 2:50pm CS329Z Lecture	Tue6 no events	Wed7 1:30pm – 2:50pm CS329Z Lecture 3:00pm – 3:30pm Coffee with John	Thu8 no events	Fri9 10:00am – 11:00am meeting	Sat10 no events
--------------------------------------	--	--------------------------------------	---	--------------------------------------	---	---------------------------------------



Agents

```
1 # RLMs are able to handle long context prompts
2 rows = json.load(open("data/emails.json"))
3 inbox = "\n\n====\n\n".join(row["email"] for row in rows)
4
5 rlm = dspy.RLM("inbox, question -> event_names: list[str], dates: list[str]", max_iters=8)
6 result = rlm(inbox=inbox, question="Which events are held in the Gates building, and on what dates?")
7 print(result.event_names)
8 print(result.dates)
```

✓ 9.1s

```
['The thesis defense for Wei Zhang']
```

```
['Monday, December 7, 2026']
```



React

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search`[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search`[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search`[Front Row (software)]

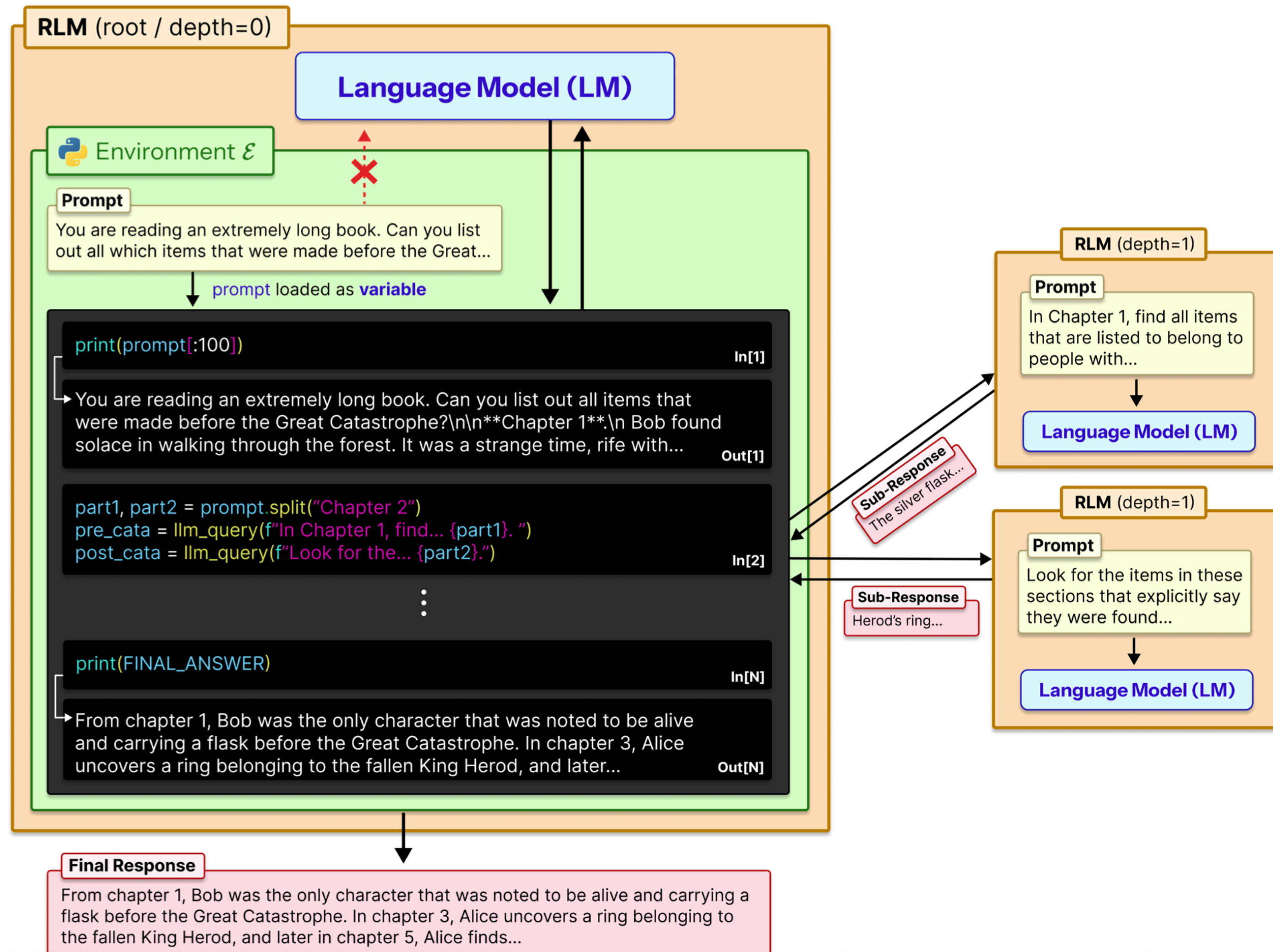
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish`[keyboard function keys]



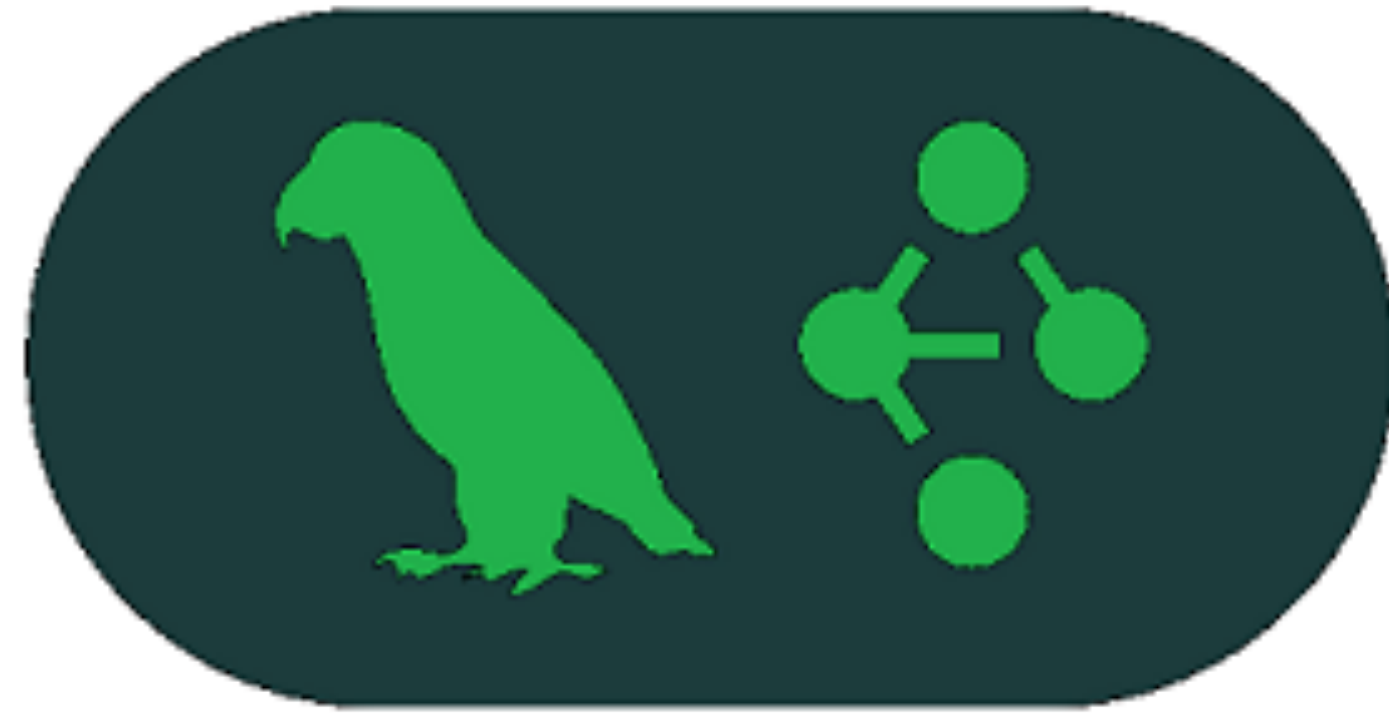
RLM



DSPy Summary

- Signatures define input/output contracts
- Modules compose and specify how the signature contract should be fulfilled
 - `dspy.Predict`, `dspy.ChainOfThought`, `dspy.BestOfN`, ...
- Optimizers can improve the prompts/weights of a program given data and a metric.
- DSPy has built in modules to enable agents to fulfill the signature.
 - `dspy.ReAct`, `dspy.RLM`





LangGraph



Outline of a LangGraph Workflow

The **State** is a shared workspace for the graph. You can think of it as an object that the graph will modify.



Outline of a LangGraph Workflow

The **State** is a shared workspace for the graph. You can think of it as an object that the graph will modify.



Start

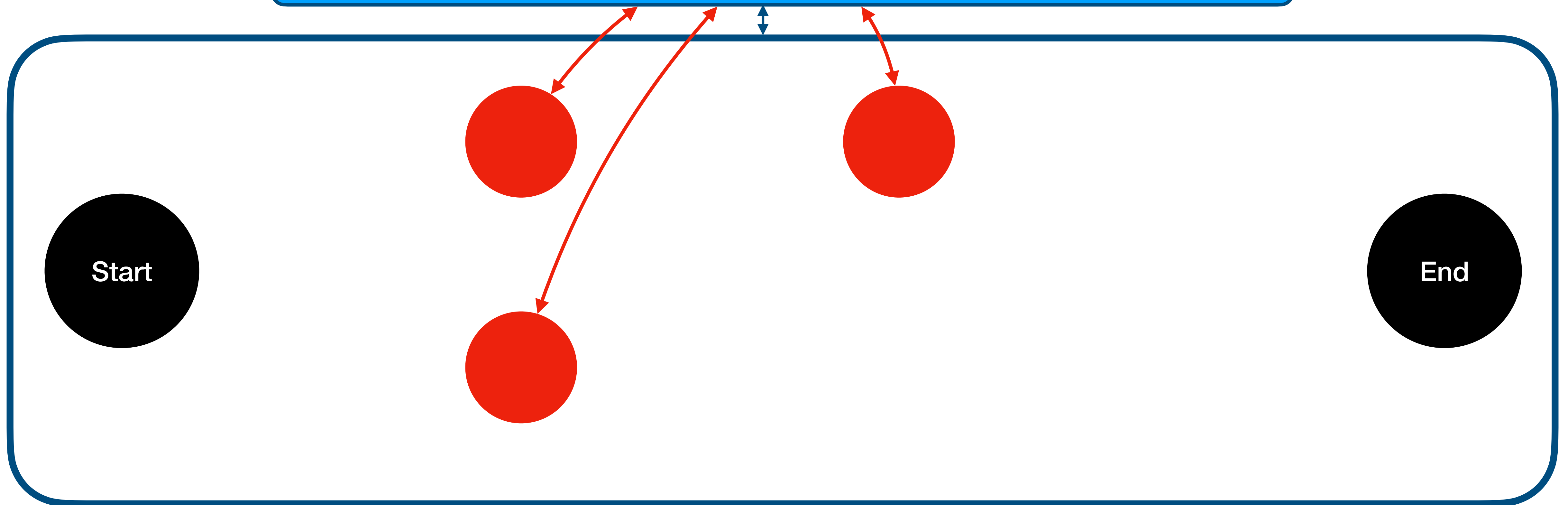
Every Graph needs a Start and End Node

End



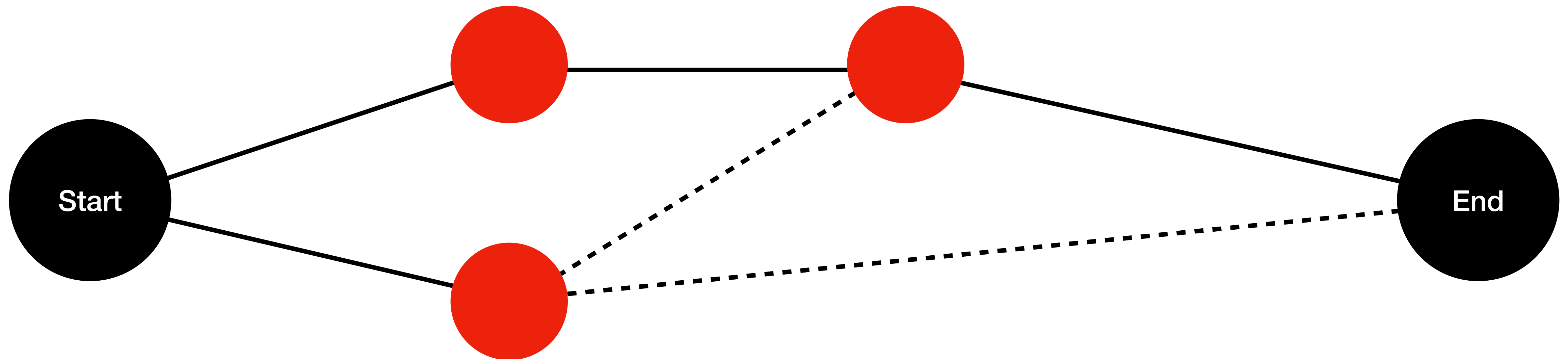
Outline of a LangGraph Workflow

The **State** is a shared workspace for the graph. You can think of it as an object that the graph will modify.



Outline of a LangGraph Workflow

The **State** is a shared workspace for the graph. You can think of it as an object that the graph will modify.



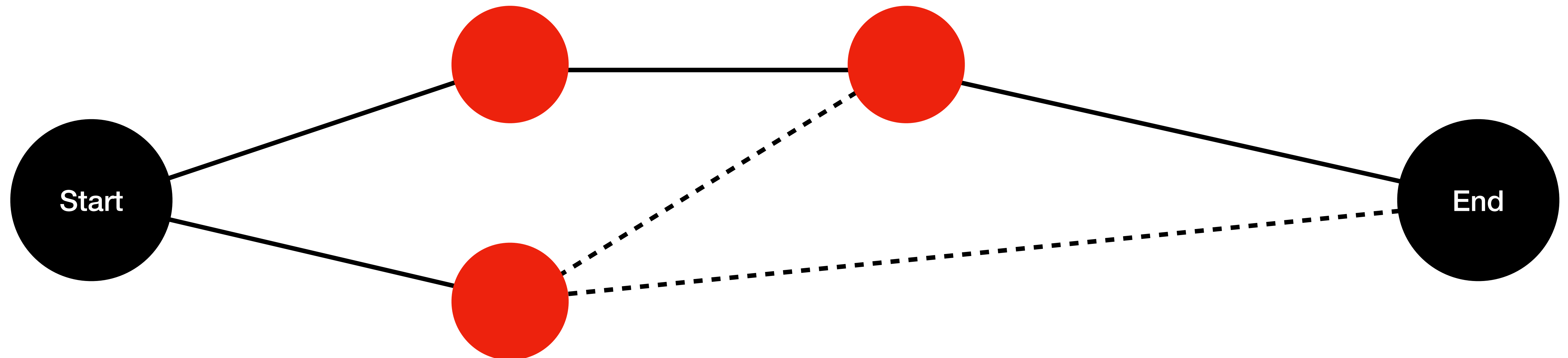
Edges Define Control Flow

Some Edges are Conditional



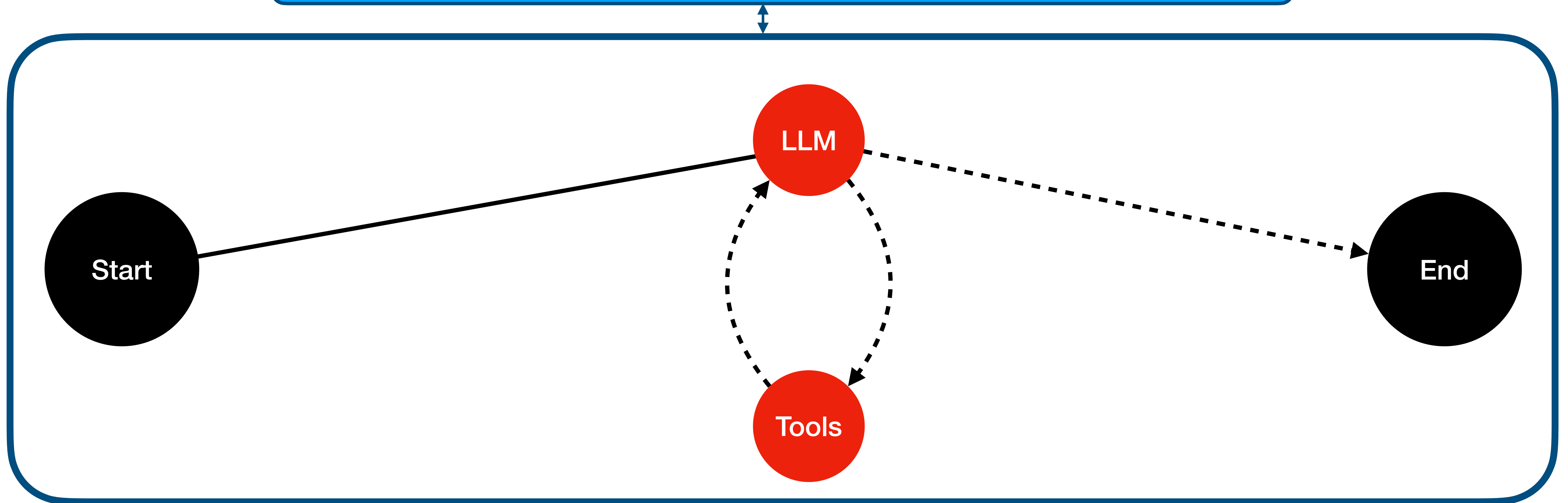
Outline of a LangGraph Workflow

The **State** is a shared workspace for the graph. You can think of it as an object that the graph will modify.



Outline of a LangGraph Agent

The **State** (can be message history)



Live Programming Demo



Building the Graph



Structured I/O

```
1 # Structured Output is Possible in LangGraph too.
2
3 from pydantic import BaseModel, Field
4
5 class ExtractEvent(BaseModel):
6     """Extract event details from an email."""
7     event_name: str | None = Field(description="short title for the event, e.g. 'Dentist' or 'Meeting'".
8     date: dt.date | None
9     start: dt.time | None = Field(description="24-hour clock")
10    duration_minutes: int | None = Field(description="duration of the event in minutes, if known")
11
12    extract_llm = langchain_llm.with_structured_output(ExtractEvent)
```

✓ 0.0s



State

```
1  # First we define the state
2  from typing import TypedDict
3
4  class State(TypedDict):
5      |     email: str
6      |     details: ExtractEvent | None
```

✓ 0.0s



Node

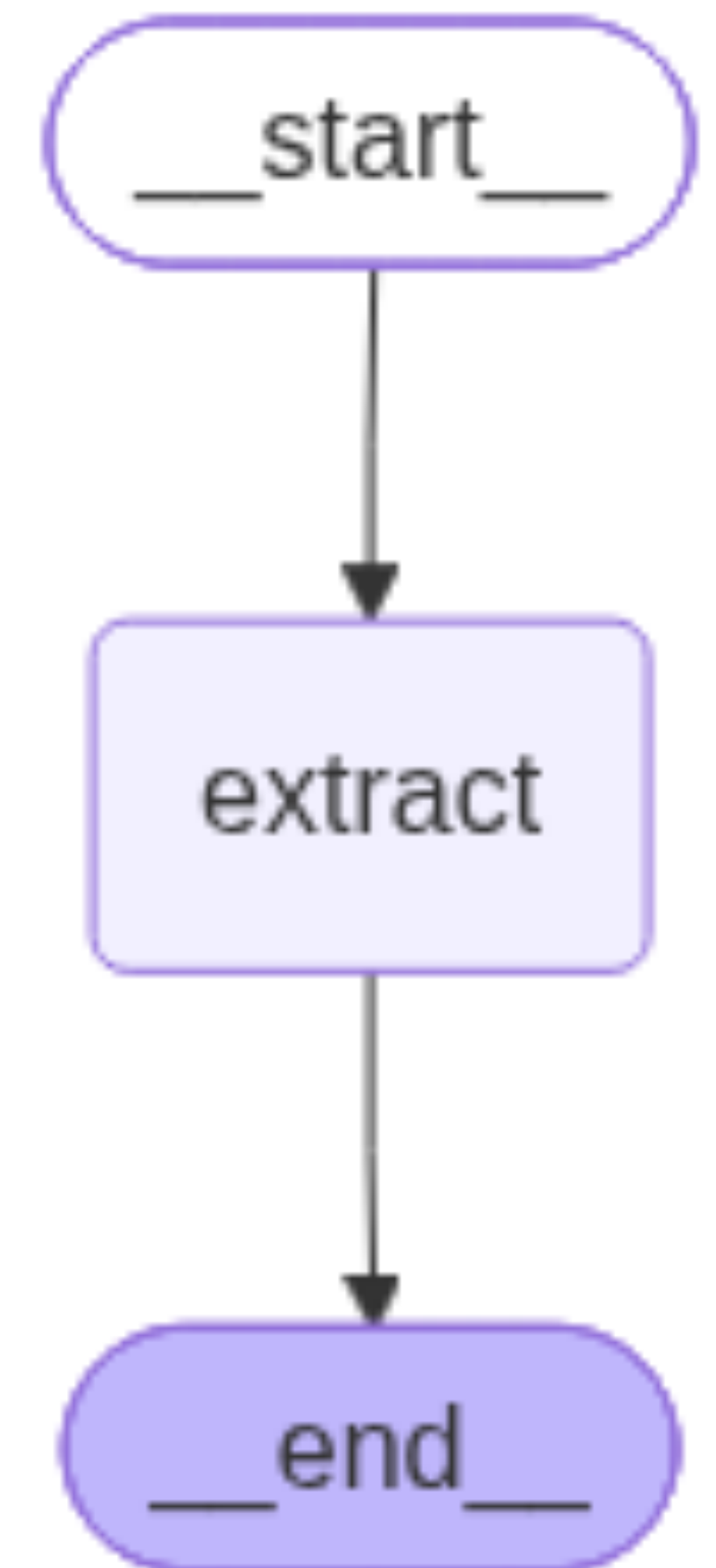
```
1  # Next: a node that reads the state, performs some computation,  
2  #       and returns the part you changed  
3  def extract(state: State):  
4  |      return {"details": extract_llm.invoke(state["email"])}  
✓ 0.0s
```



Graph

```
1 from langgraph.graph import StateGraph, START, END
2
3 # Build the Graph
4 builder = StateGraph(State)
5 builder.add_node("extract", extract)
6 builder.add_edge(START, "extract")
7 builder.add_edge("extract", END)
8 graph = builder.compile()
```

✓ 0.1s



Graph

```
1 # Invoke the graph
2 graph.invoke({"email": "We have a meeting scheduled for this Friday, October 9th at 10 AM PST."})
```

✓ 0.0s

```
{'email': 'We have a meeting scheduled for this Friday, October 9th at 10 AM PST.',
 'details': ExtractEvent(event_name='Meeting', date=datetime.date(2026, 10, 9), start=datetime.time(10, 0, tzinfo=TzInfo(0)),
```



Conditional Edges and Workflows



State

```
1  # Define the State
2  class CalendarState(State):
3      |      emoji: str
4      |      status: str
✓ 0.0s
```



Nodes

```
1 # Create Nodes
2 def pick_emoji(state: CalendarState):
3     reply = langchain_llvm.invoke(f"Reply with exactly one emoji character, nothing else, for a calendar event ca
4     return {"emoji": reply.text.strip()}
5
6 def add_to_calendar(state: CalendarState):
7     d = state["details"]
8     tinycal.add_event(d.event_name, d.date, d.start, duration_minutes=d.duration_minutes, emoji=state["emoji"])
9     return {"status": "added to calendar"}
```



Conditional Edge

```
1 # Create a Conditional Edge
2 from typing import Literal
3
4 def route(state: CalendarState) -> Literal["pick_emoji", "__end__"]: # plain Python decides where to go next
5     d = state["details"]
6     return "pick_emoji" if d.event_name and d.date and d.start else END
```

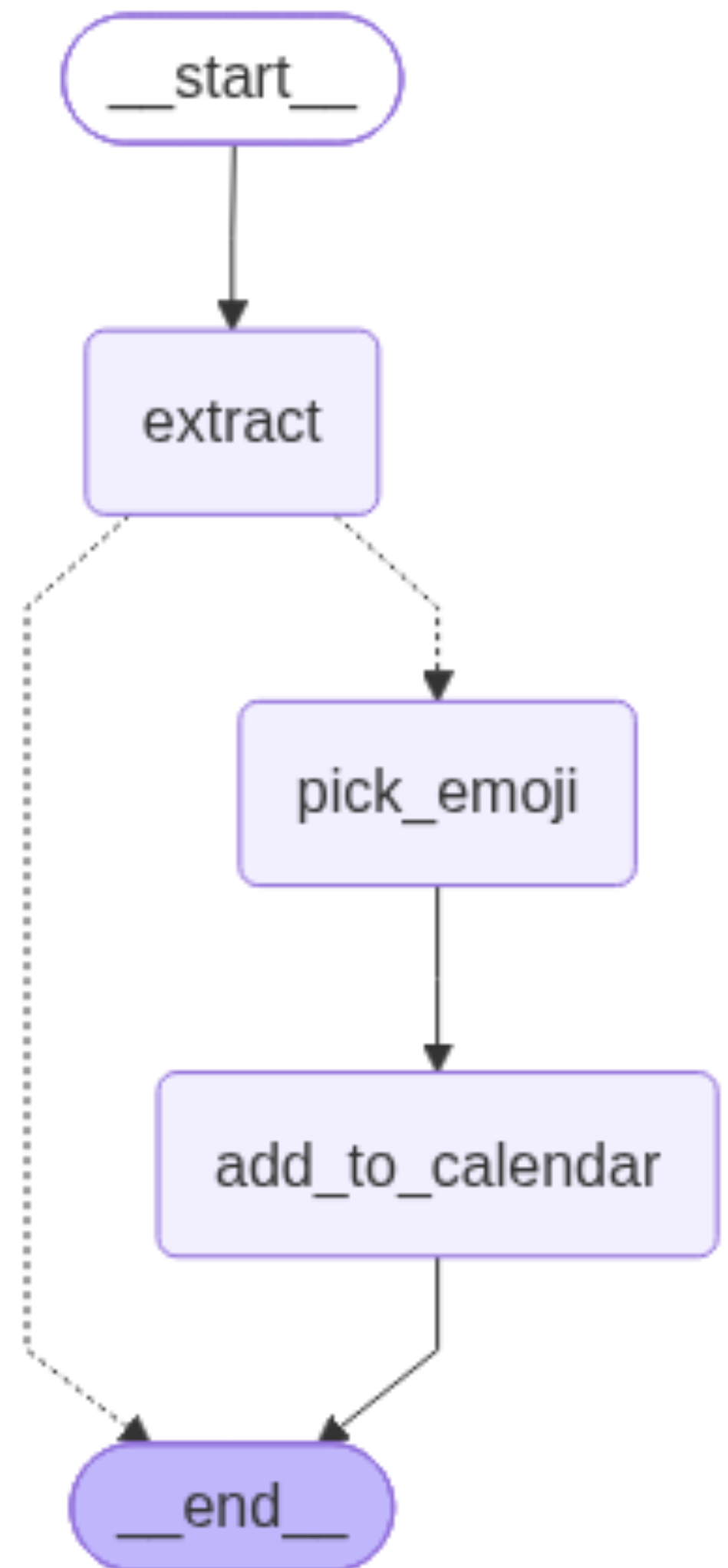
✓ 0.0s



Graph

```
1  # Build the graph
2  builder = StateGraph(CalendarState)
3  builder.add_node("extract", extract)
4  builder.add_node("pick_emoji", pick_emoji)
5  builder.add_node("add_to_calendar", add_to_calendar)
6
7  builder.add_edge(START, "extract")
8  builder.add_conditional_edges("extract", route)
9  builder.add_edge("pick_emoji", "add_to_calendar")
10 builder.add_edge("add_to_calendar", END)
11
12 update_calendar = builder.compile()
```

✓ 0.0s



Graph

```
1 update_calendar.invoke({"email":"Dentist Appointment Confirmed, Wedndesday October 7th at 11am."})
```

✓ 1.7s

```
{'email': 'Dentist Appointment Confirmed, Wedndesday October 7th at 11am.',  
 'details': ExtractEvent(event_name='Dentist Appointment', date=datetime.date(2026, 10, 7), start=datetime.time(11,  
 'emoji': '🦷',  
 'status': 'added to calendar'}
```

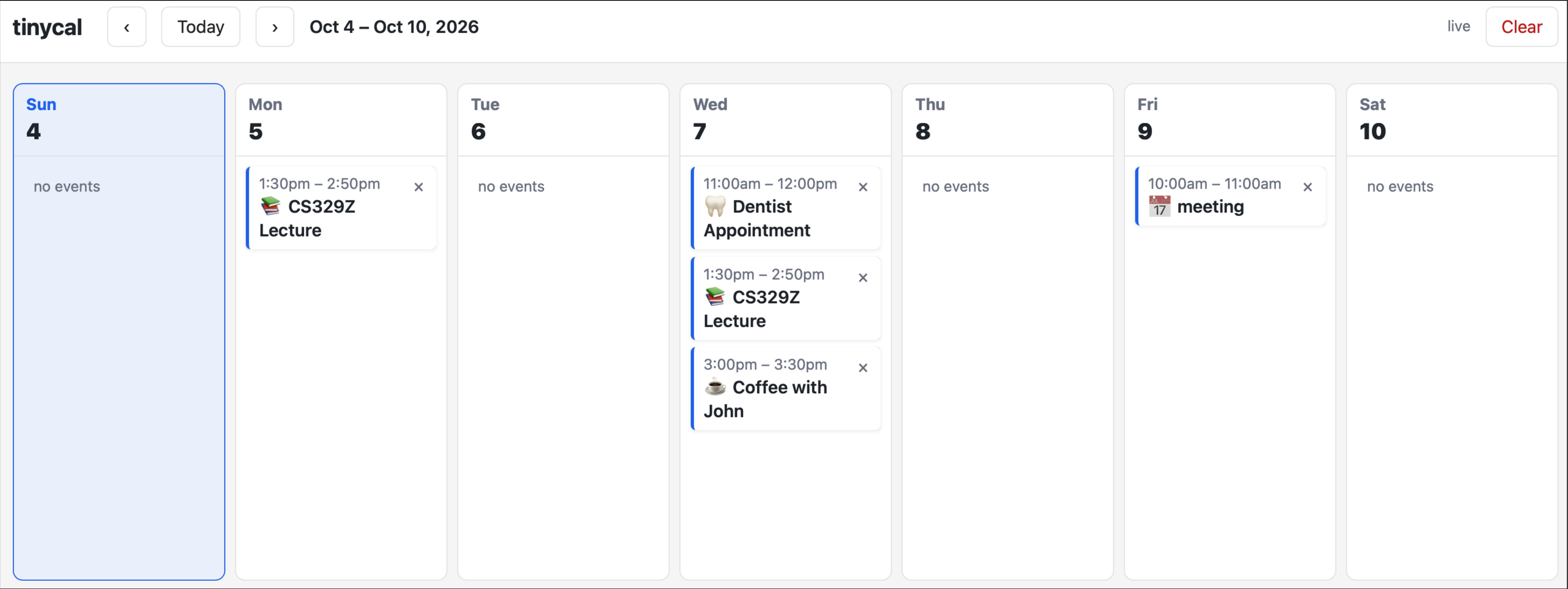
```
1 update_calendar.invoke({"email":"CS329Z Open Router Credits have arrived! Please check your inbox."})
```

✓ 0.0s

```
{'email': 'CS329Z Open Router Credits have arrived! Please check your inbox.',  
 'details': ExtractEvent(event_name=None, date=None, start=None, duration_minutes=None)}
```



Graph



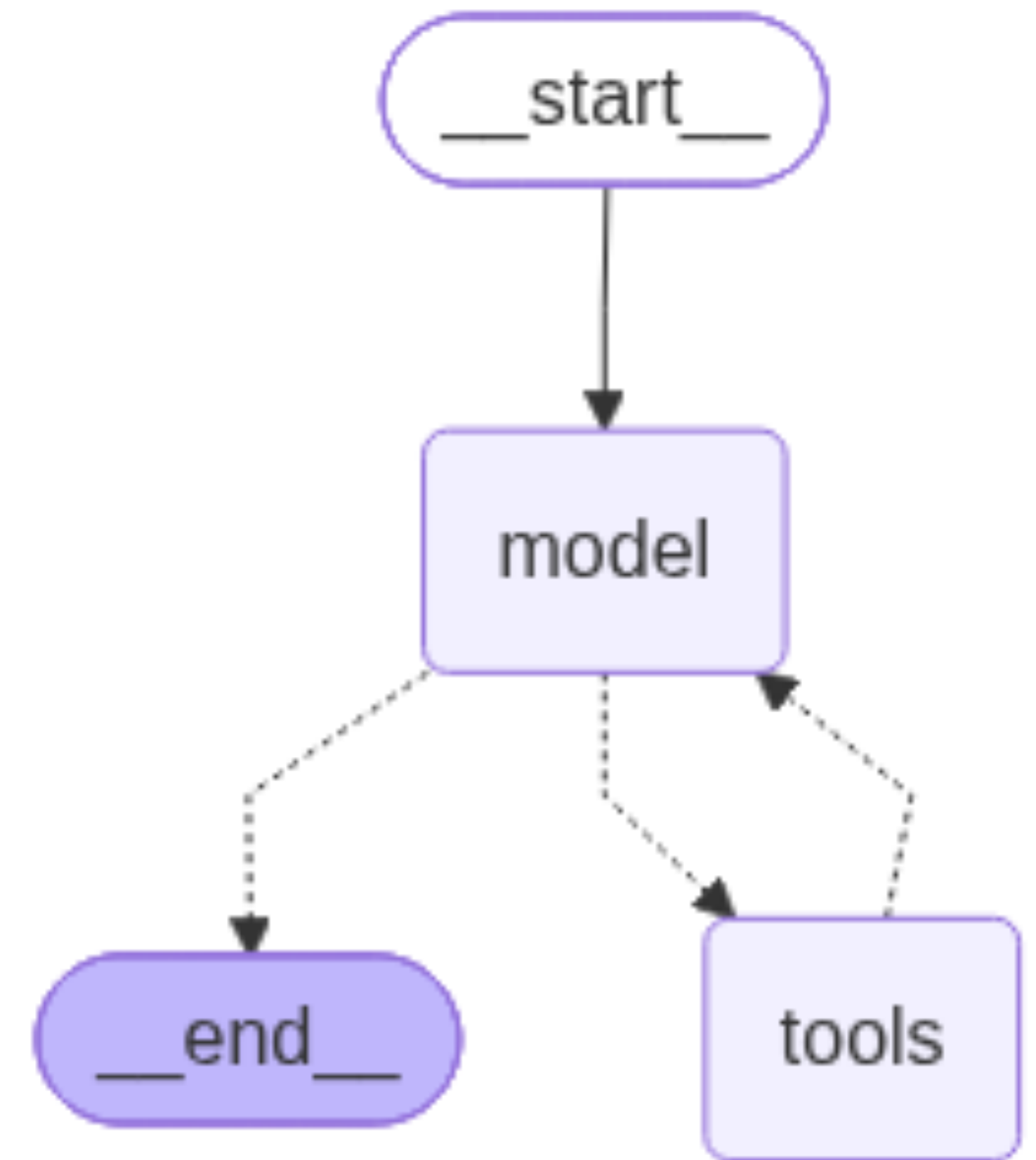
Agents



Agents

```
1 # Similar to DSPy we can make an agent in two lines
2 from langchain.agents import create_agent
3 langchain_cal_agent = create_agent(langchain_llm, tools=tinycal.TOOLS)
```

✓ 0.1s



Agents

```
1 langchain_cal_agent.invoke({"messages": [("user", "Please move my coffee with John on Oct 7th to be 30 minutes later")]})
```

✓ 2.7s

tinycal < Today > Oct 4 – Oct 10, 2026 live Clear

Sun 4	Mon 5	Tue 6	Wed 7	Thu 8	Fri 9	Sat 10
no events	1:30pm – 2:50pm × 📖 CS329Z Lecture	no events	11:00am – 12:00pm × 🦷 Dentist Appointment 1:30pm – 2:50pm × 📖 CS329Z Lecture 3:30pm – 4:00pm × ☕ Coffee with John	no events	10:00am – 11:00am × 🇺🇸 17 meeting	no events



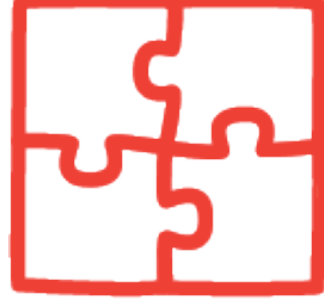
LangGraph Summary

- LangGraph operates on shared state over a Graph
- Nodes read and write to the state
- Edges define the control flow between the nodes
- Some edges can be conditional (to enable branching workflows)
- LangGraph has built-in tooling to create agents `create_agent`



Recap

- **Workflows compose model calls, tools, and program logic into a larger AI system.**
- **Frameworks expose different abstractions:**
DSPy uses signatures and modules;
LangGraph uses state, nodes, and edges.
- **We design the control flow and choose where the model makes decisions.**
- **Improvement requires data and evaluation.**
DSPy makes this explicit through optimizers.



DSPy



LangGraph



LlamaIndex



crewai

...



Open Source Agent Harnesses

“Engineer” Agents?

Workflows

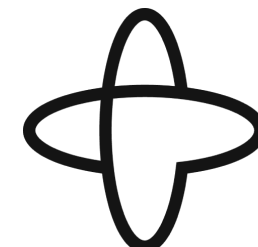
Agent Harnesses

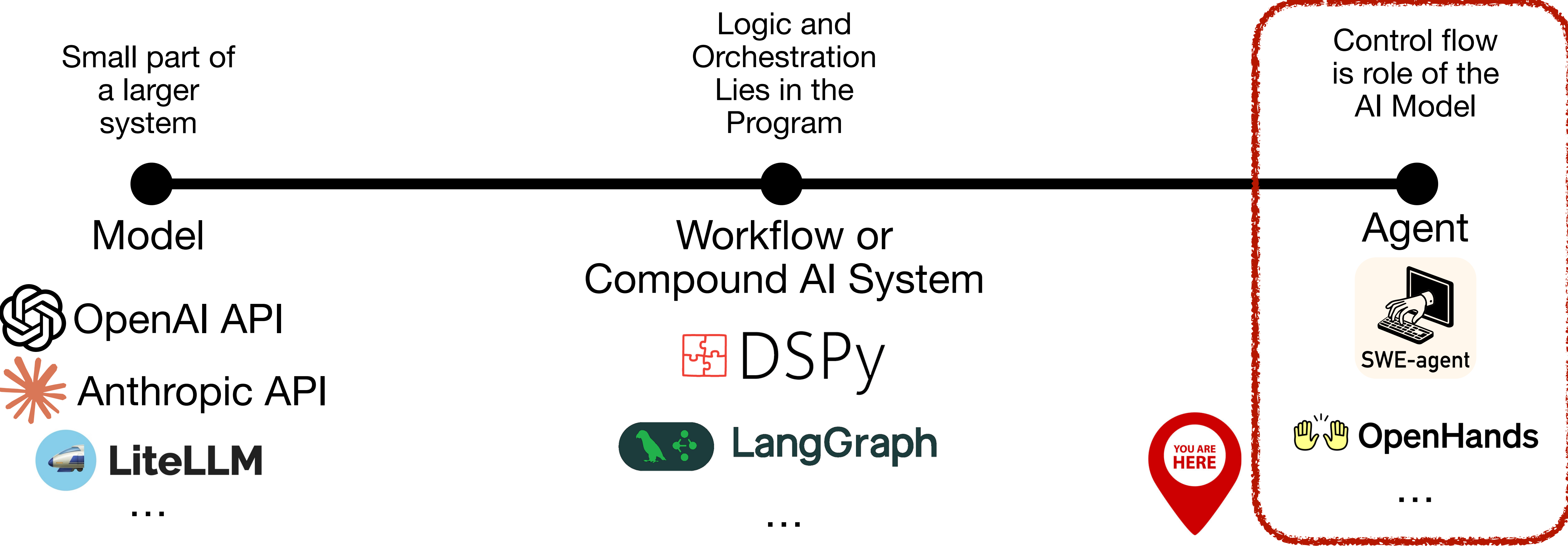
Orchestration

Best Practices

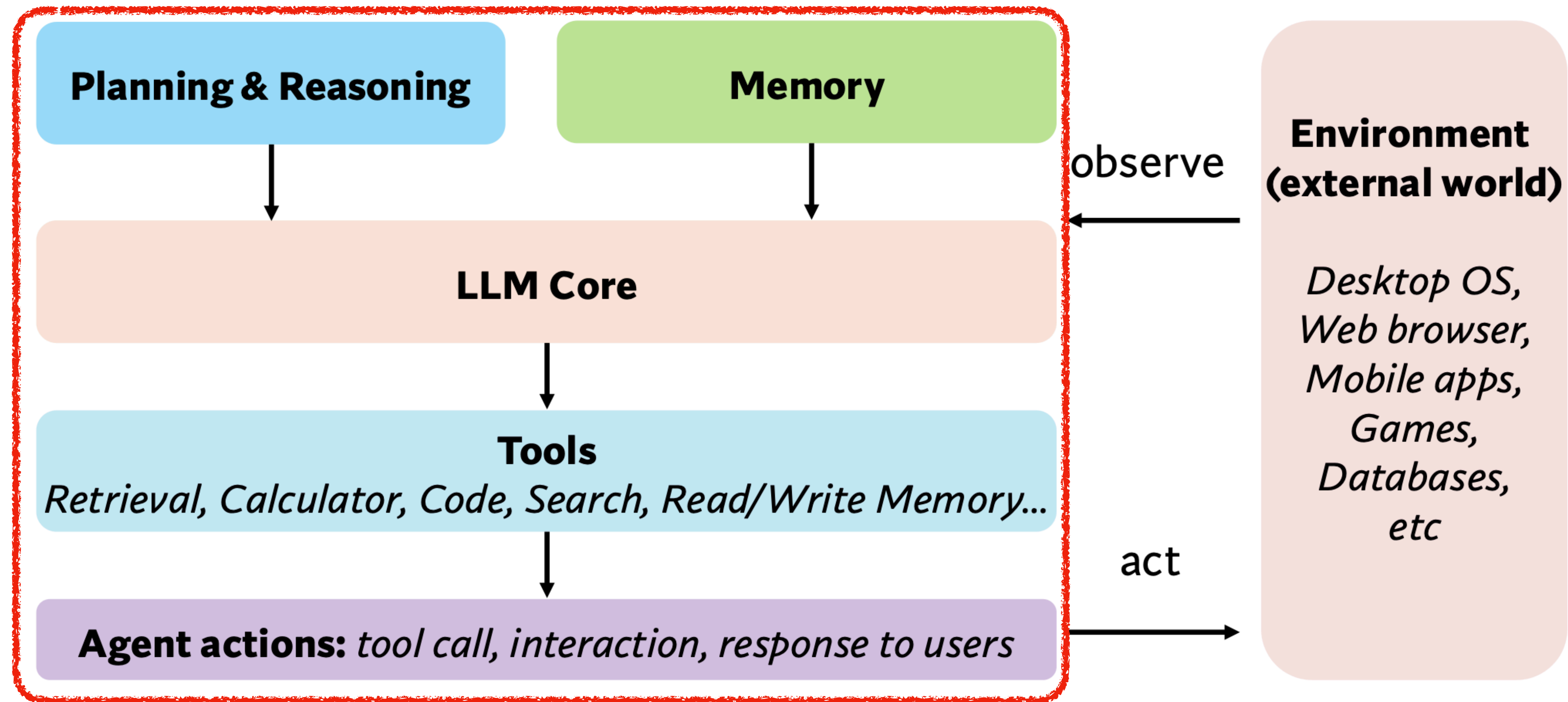


Framework Landscape

Reliability:  Temporal, ... **Observability:**  Langfuse, *mlflow*, ...



What is an Agent Harness?



What differentiates Agent Harnesses?

- **Building an agent involves design choices:** tools, memory, context management, and execution.
- **Different harnesses make different tradeoffs** to suit their tasks and operating environments.
- **A good harness balances task success, cost, latency, and user control.** The right balance depends on the use case.



Mini SWE-Agent



- Does not have any tools other than bash
- Has a completely linear history — every step of the agent just appends to the messages and that's it.
- Executes actions with `subprocess.run` — every action is completely independent (as opposed to keeping a stateful shell session running)



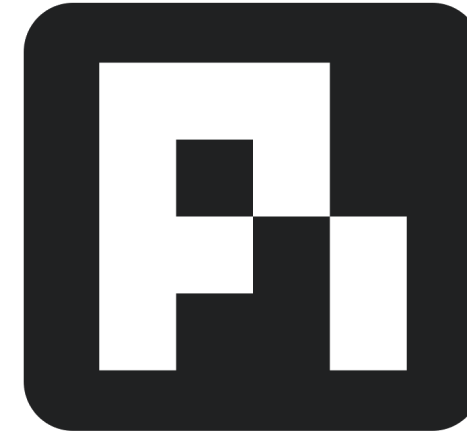
Open Hands



- **Stores every step as an event** — one JSON file per event on disk, and the messages sent to the model are rebuilt from them each step.
- **Keeps a persistent terminal** — a tmux session, so cd, environment variables and virtualenvs carry over between commands
- **Stops itself when it is stuck** — a detector ends the run after 4 repeats of the same action and result.



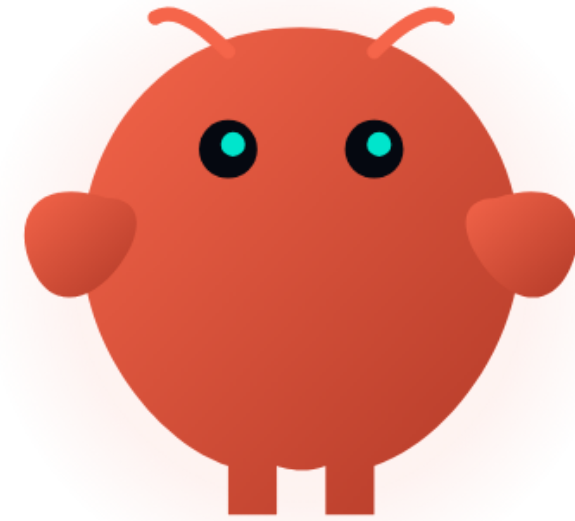
Pi Agent



- **Has only four tools** — read, write, edit, and bash, with a system prompt under 1,000 tokens.
- **No permission system** — it runs with whatever permissions you launched with.
- **Stores history as a tree, not a list** — you can go back to an earlier message and branch from there
- **Deliberately Simple** — there is no plan mode and no sub-agent tool



Open Claw

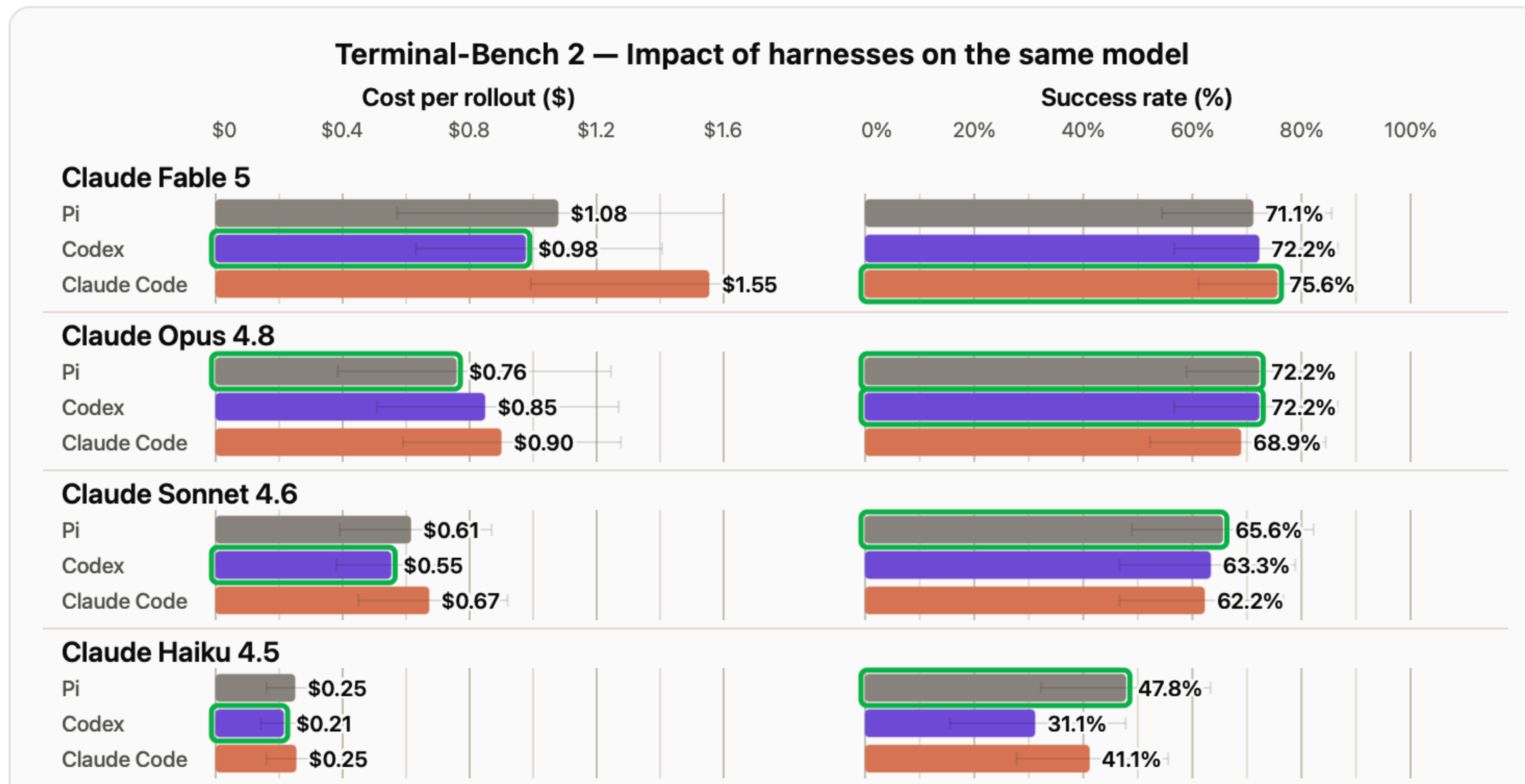


OpenClaw

- **Wakes itself up every 30 minutes** — a heartbeat turn runs on a schedule, and the agent replies NO_REPLY to stay silent.
- **Runs as one long-lived Gateway** — a single process owns all the chat channels.
- **Builds its identity from Markdown files** — AGENTS.md, SOUL.md, IDENTITY.md, USER.md and MEMORY.md are injected into the system prompt
- **Stores memories every night** — at 3 AM a background job reads through the day's notes and moves the parts worth keeping into MEMORY.md



The Harness can make a big difference



Meta-Harness

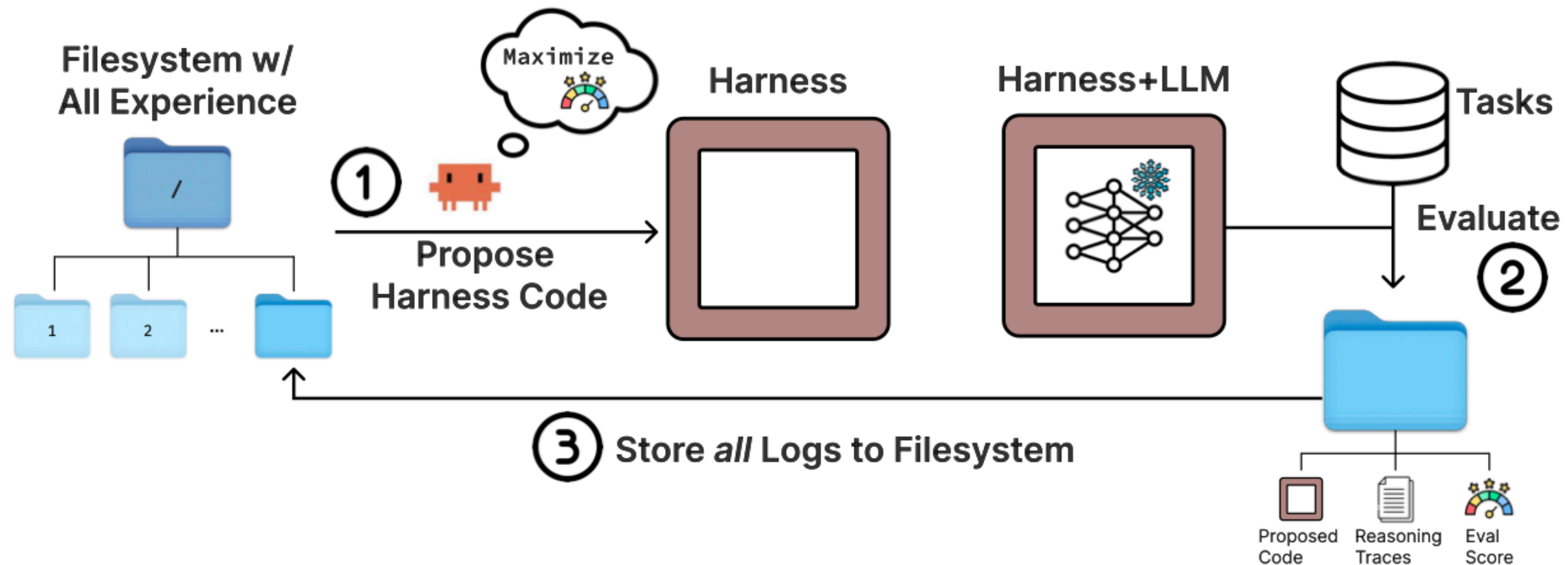


Figure 2: **Meta-Harness search loop.** (1) An agent reads a filesystem containing all prior candidates' source code, execution traces, and scores, and proposes a new harness. (2) We evaluate the proposed harness on evaluation tasks. (3) All logs (proposed code, reasoning traces, evaluation scores) are stored in the filesystem in a new directory, and the loop repeats.



Meta-Harness

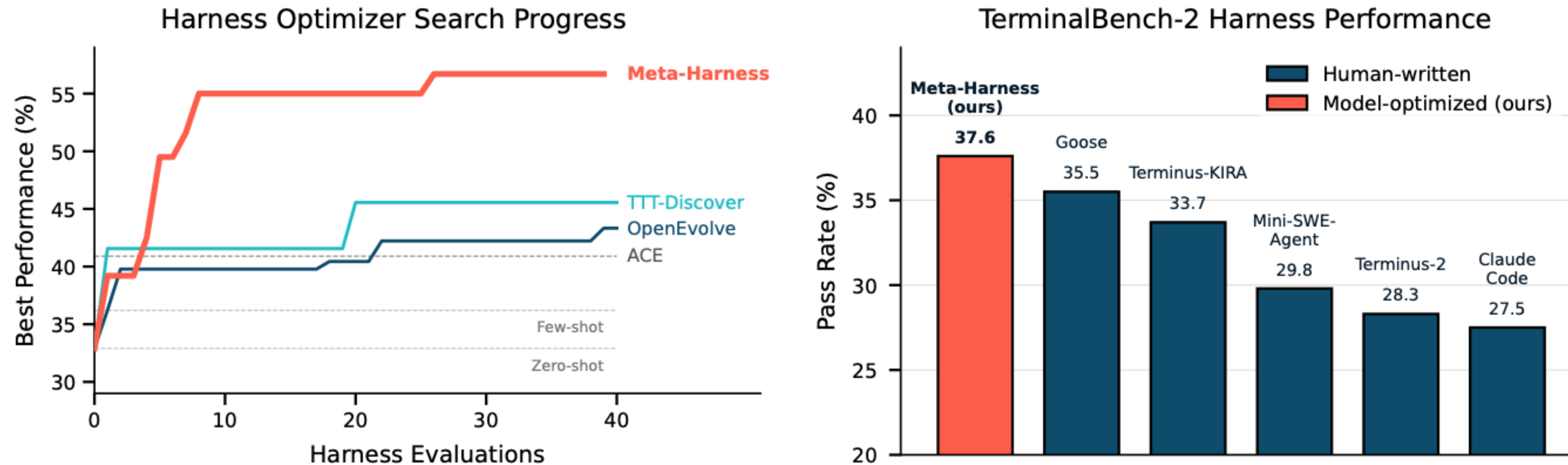


Figure 1: **(Left)** On text classification, Meta-Harness outperforms the best prior hand-designed harnesses (ACE) and existing text optimizers (TTT-Discover, OpenEvolve), matching the next-best method's final accuracy after just 4 evaluations. **(Right)** On TerminalBench-2, Meta-Harness outperforms all reported Claude Haiku 4.5 harnesses.



Bringing it together

- There is no one way to build an agent.
- In this course you will be experimenting with all of these design decisions.
- Perhaps you can build the next popular agent harness :D



Orchestration and Telemetry

“Engineer” Agents?

Workflows

Agent Harnesses

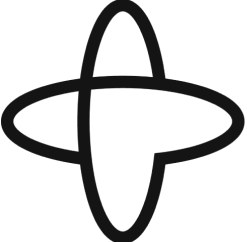
Orchestration

Best Practices



Framework Landscape

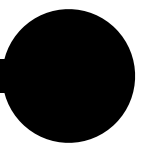
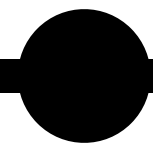
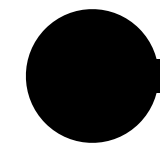


Reliability:  Temporal, ... **Observability:**  Langfuse, *mlflow*, ...

Small part of
a larger
system

Logic and
Orchestration
Lies in the
Program

Control flow
is role of the
AI Model



Model

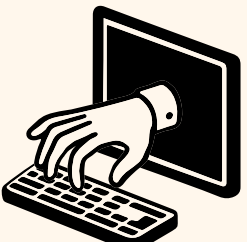
Workflow or
Compound AI System

Agent

 OpenAI API
 Anthropic API
 **LiteLLM**
...

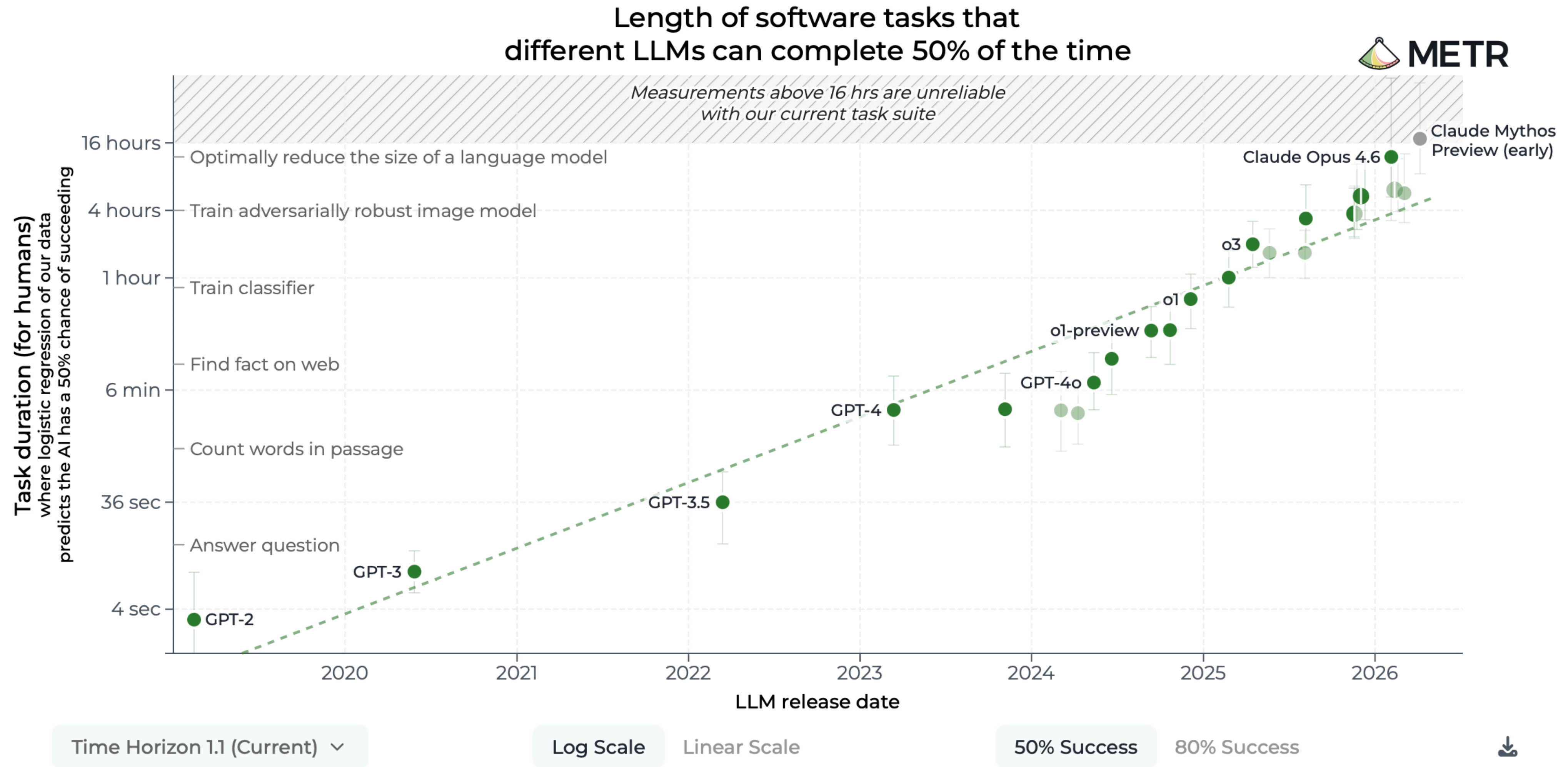
 DSPy

 **LangGraph**

 SWE-agent

 **OpenHands**
...

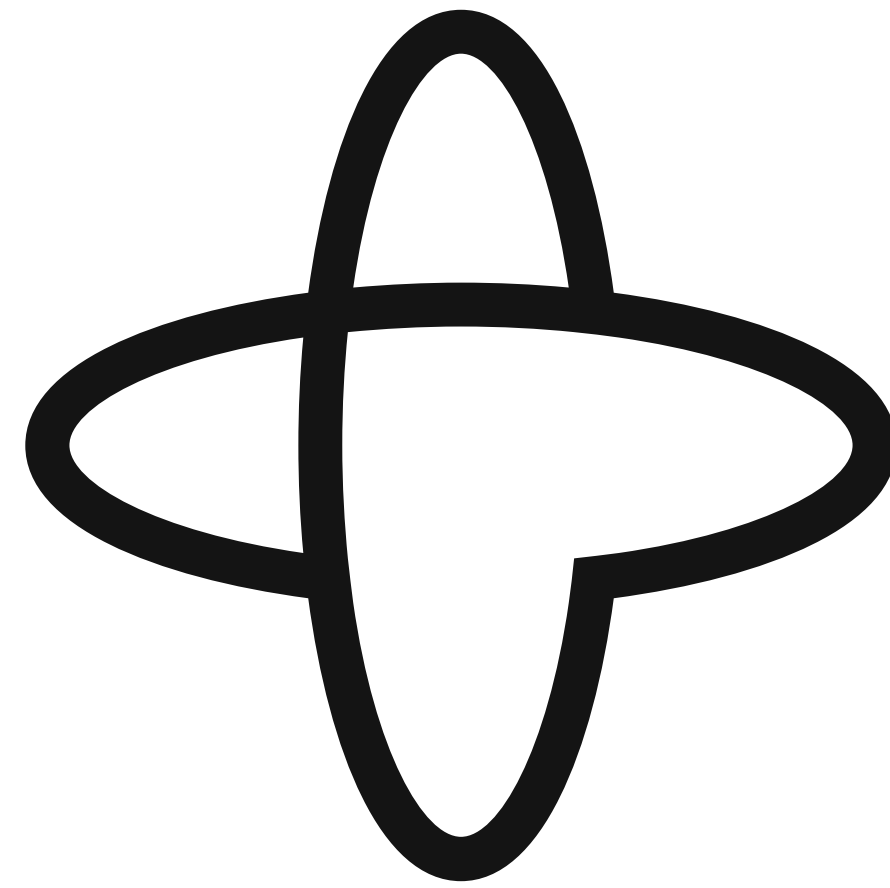
Why is orchestration/telemetry needed?



Why is orchestration/telemetry needed?

- Agents run for long periods of time (although previous slides was “human-time”), and the infra supporting them must be reliable.
- It is useful to have logs and observability tools for debugging.
- These are hard taught lessons from good software engineering practices that transfer over to agents.



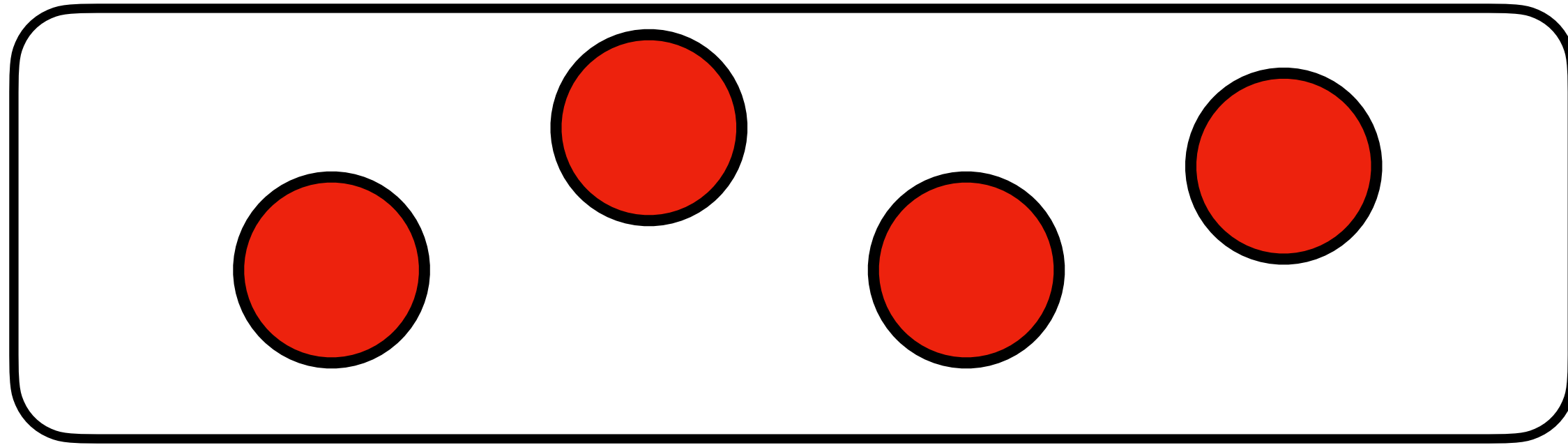


Temporal



Temporal Execution Model

Your Code (Client)

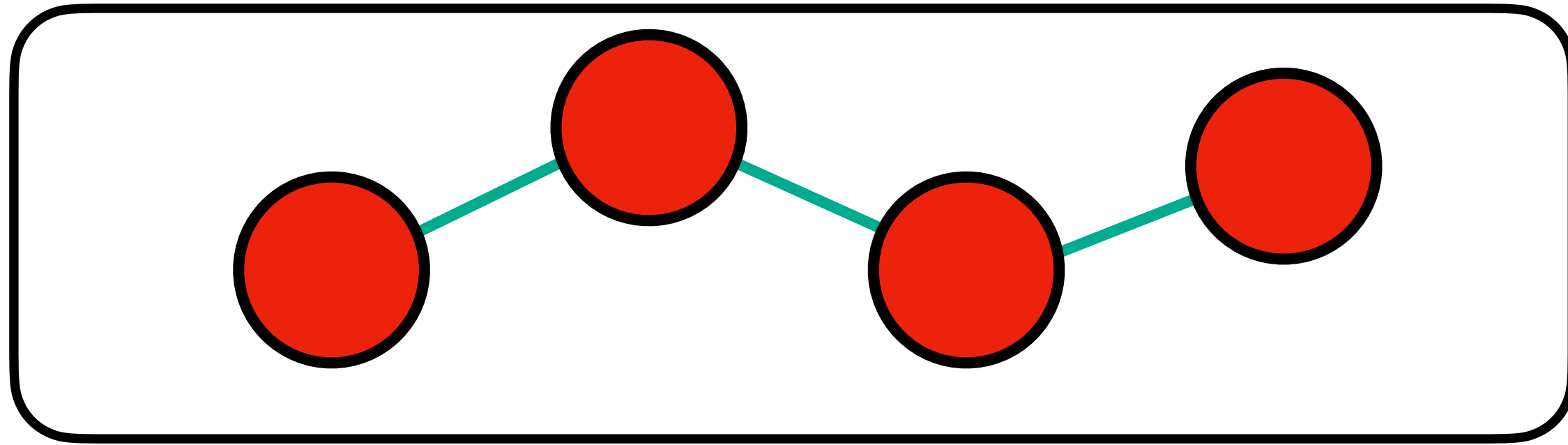


Activities are functional units that can fail.



Temporal Execution Model

Your Code (Client)

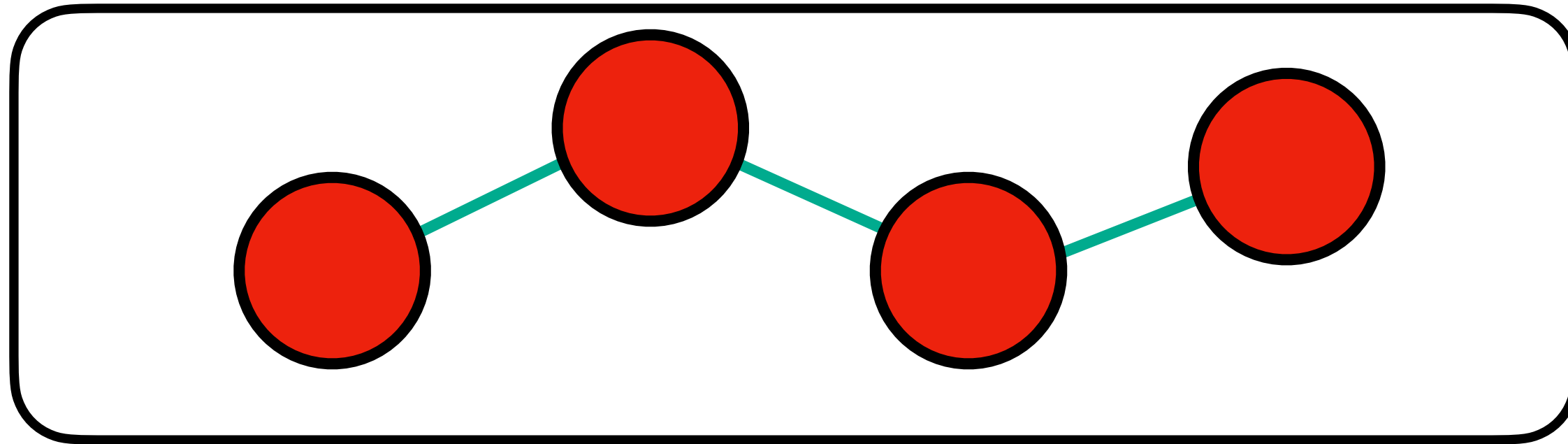


Activities are functional units that can fail.
Workflows link activities together.

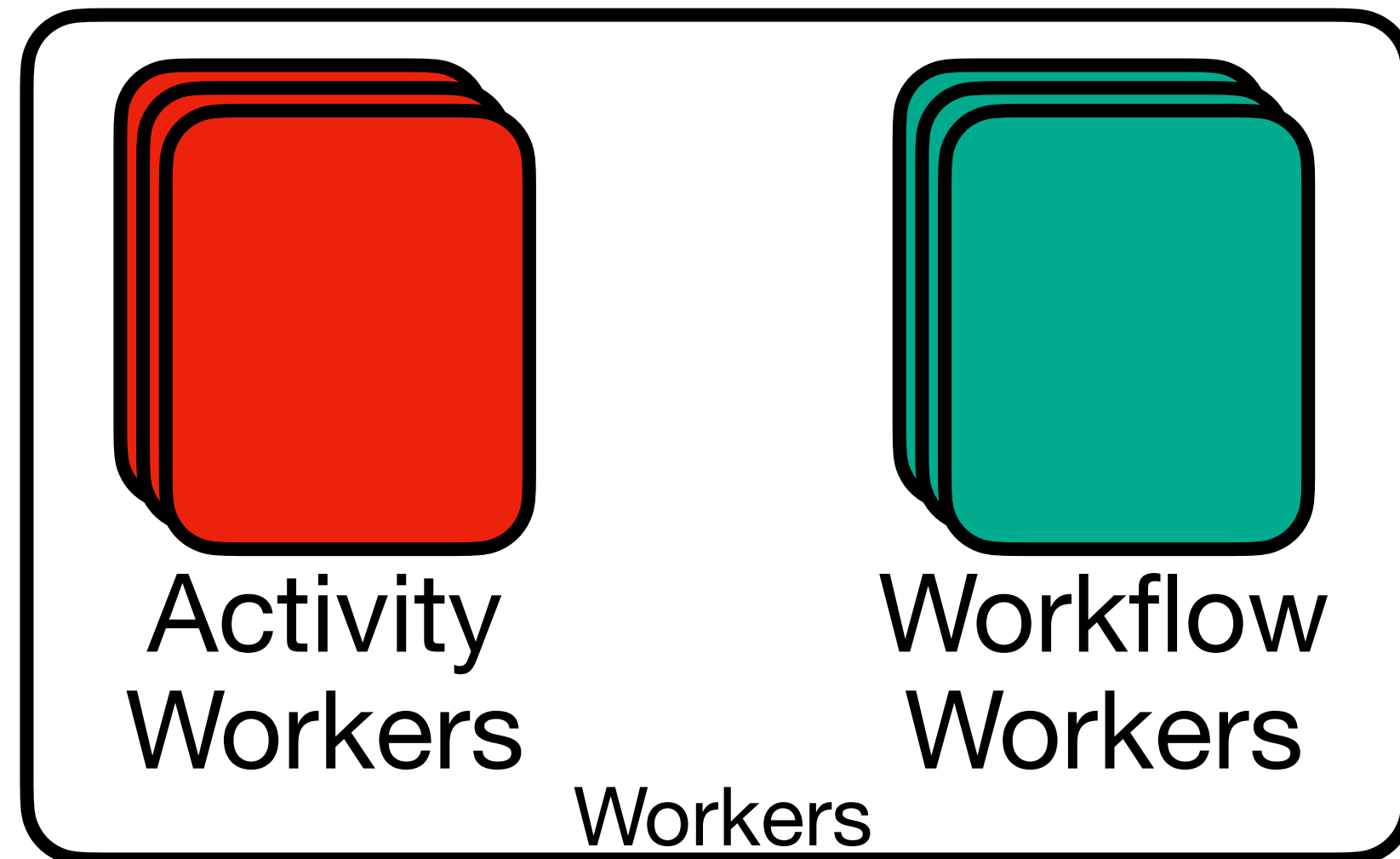


Temporal Execution Model

Your Code (Client)



Activities are functional units that can fail.
Workflows link activities together.

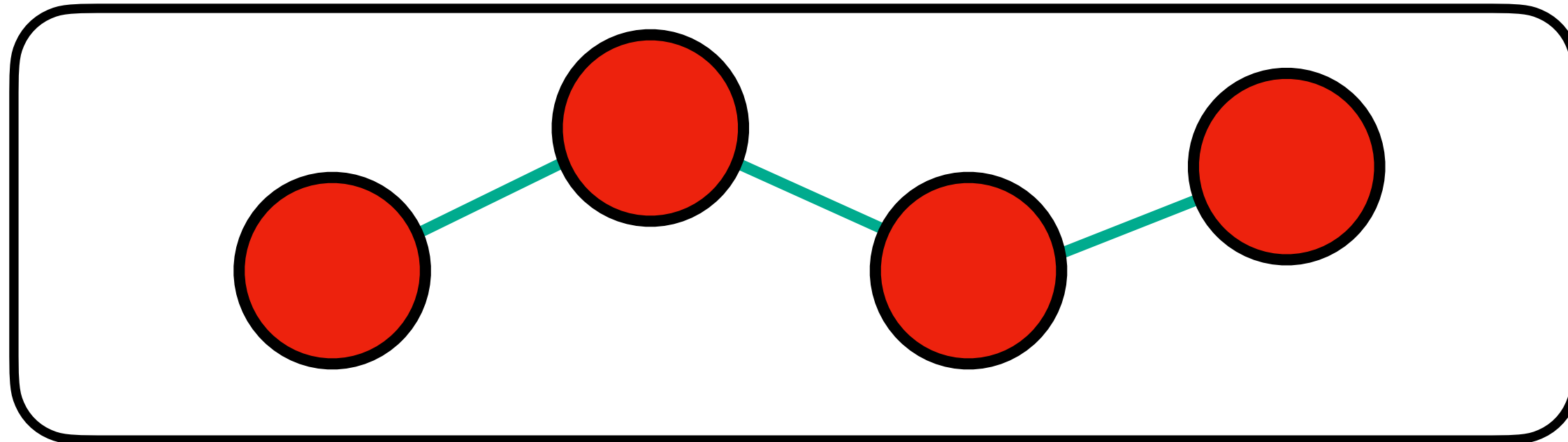


Server

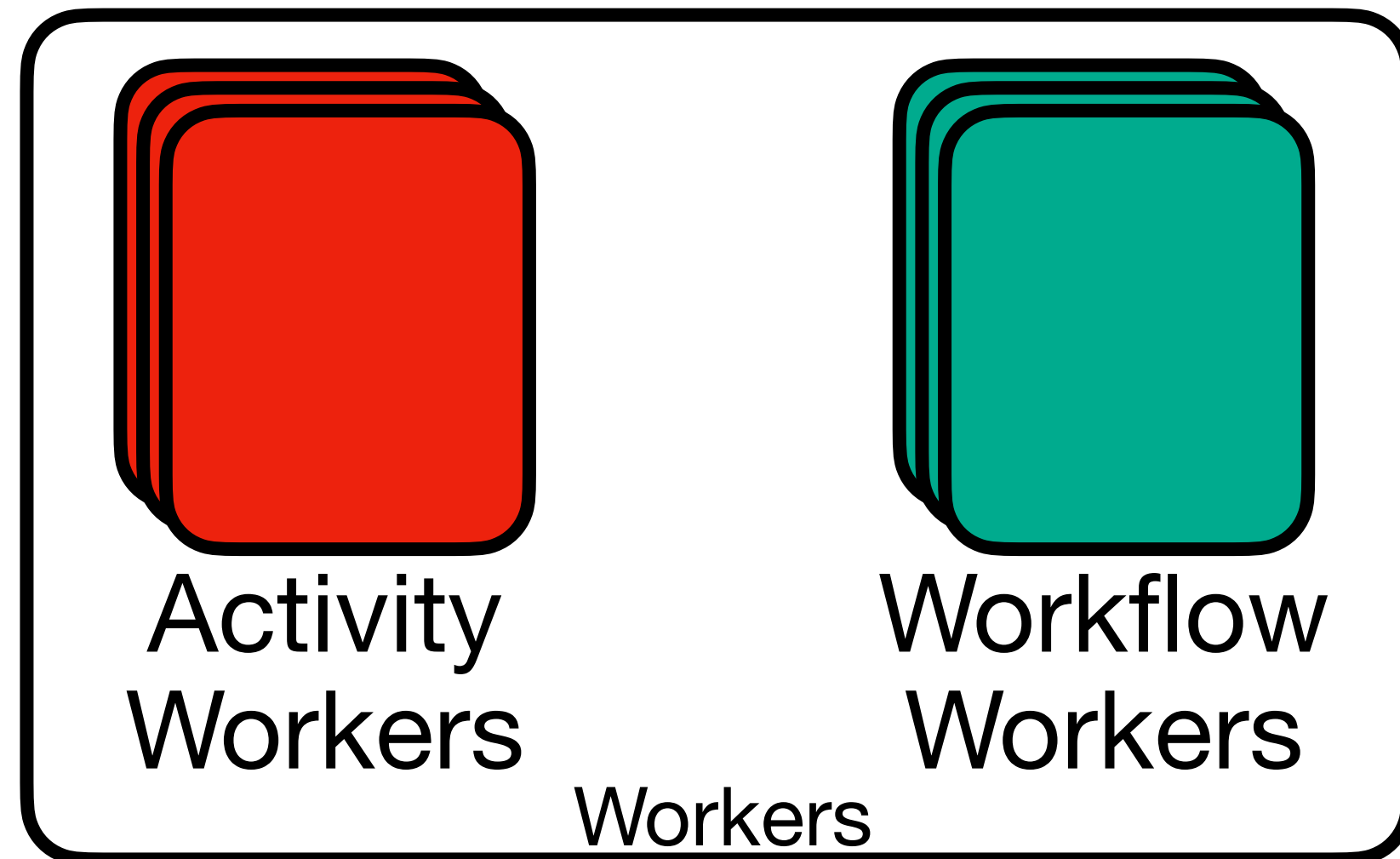


Temporal Execution Model

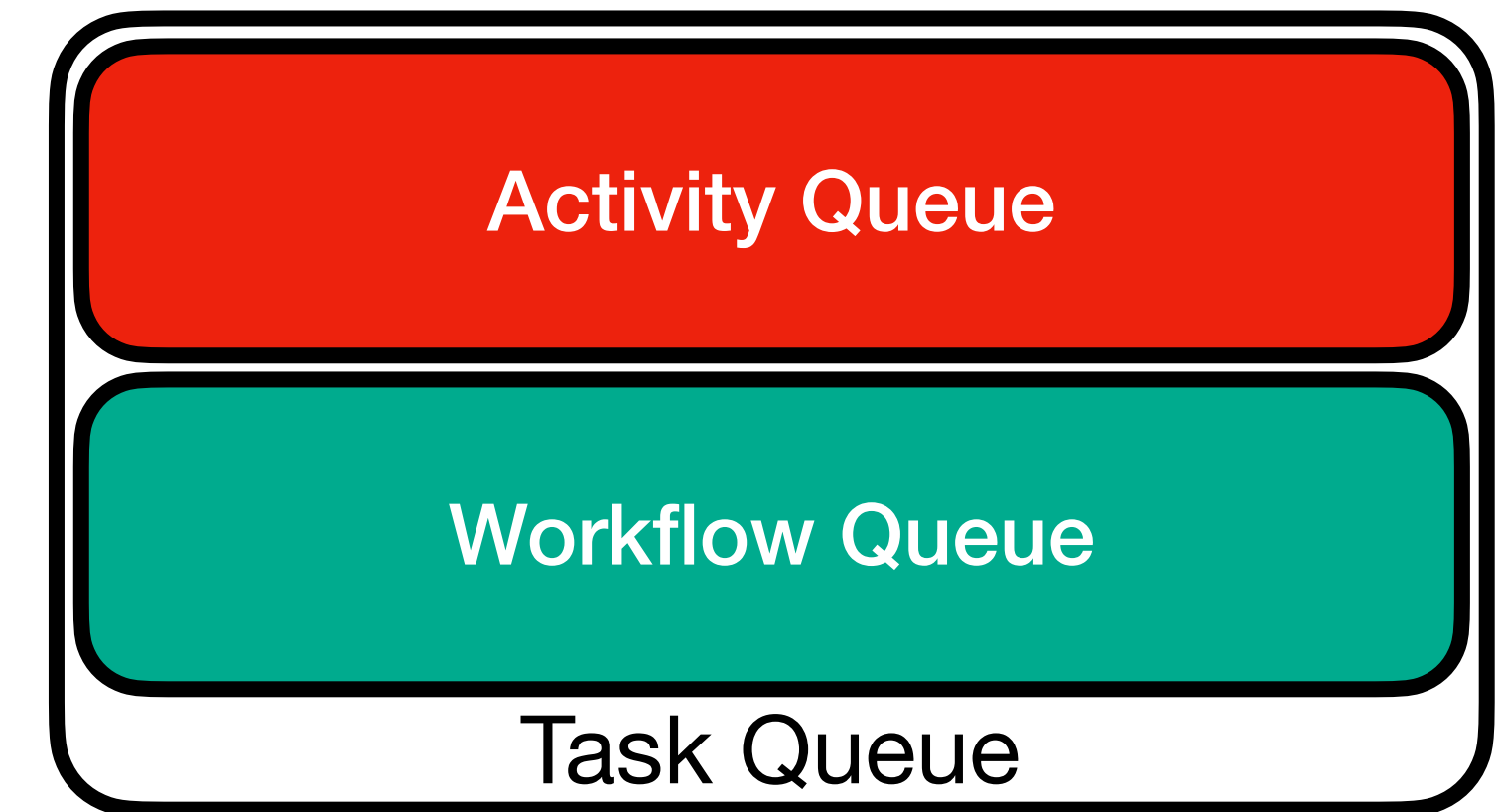
Your Code (Client)



Activities are functional units that can fail.
Workflows link activities together.



Server



Live Programming Demo



Durable Workflows



Making a Durable Workflow: Activities

```
1 # Activities are functional units that can fail
2 #     They are orchestrated by the workflow
3 #     Here we reuse our DSPy extraction and emoji selection modules
4 emoji_cot = dspy.ChainOfThought(PickEmoji)
5 @activity.defn
6 def extract_event(email: str) -> dict | None:
7     d = extract_cot(email=email)
8     if d.event_name is None or d.date is None or d.start is None:
9         return None
10    return {"title": d.event_name,
11           "date": str(d.date),
12           "start": str(d.start)[:5],
13           "duration_minutes": d.duration_minutes}
14
15 @activity.defn
16 def select_emoji(title: str) -> str:
17     return emoji_cot(event_name=title).emoji
18
19 @activity.defn
20 def add_to_calendar(details: dict, emoji: str) -> str:
21     tinycal.add_event(**details, emoji=emoji)
22     return "added to calendar"
```



Making a Durable Workflow: Workflow

```
1  # The workflow: UpdateCalendarFromEmail.forward, with execute_activity around every call that can fail.
2  # It must be deterministic (no LLM calls, no I/O of its own), because Temporal re-runs it from the record.
3  from datetime import timedelta
4  from temporalio.common import RetryPolicy
5
6  RETRY = RetryPolicy(initial_interval=timedelta(seconds=1), maximum_attempts=5)
7
8  @workflow.defn(sandboxed=False)
9  class CalendarWorkflow:
10     @workflow.run
11     async def run(self, email: str) -> str:
12         details = await workflow.execute_activity(extract_event, email,
13                                                    start_to_close_timeout=timedelta(minutes=1), retry_policy=RETRY)
14         if details is None:
15             return "no event found in email"
16         emoji = await workflow.execute_activity(select_emoji, details["title"],
17                                                  start_to_close_timeout=timedelta(minutes=1), retry_policy=RETRY)
18         return await workflow.execute_activity(add_to_calendar, args=[details, emoji],
19                                                start_to_close_timeout=timedelta(minutes=1), retry_policy=RETRY)
```



Making a Durable Workflow: Worker

```
1  # A worker is the process that actually runs workflows and activities.
2  import asyncio
3  from concurrent.futures import ThreadPoolExecutor
4
5  worker = Worker(client, task_queue="calendar",
6                 workflows=[CalendarWorkflow],
7                 activities=[extract_event, select_emoji, add_to_calendar],
8                 activity_executor=ThreadPoolExecutor(4))
9  worker_task = asyncio.create_task(worker.run())
```



Making a Durable Workflow: Run it

```
1 # Run it. Every step is recorded: open http://localhost:8233 and click on calendar-1.
2 print(await client.execute_workflow(CalendarWorkflow.run,
3                                     "Office hours are Thursday October 8th at 3pm",
4                                     id="calendar-1",
5                                     task_queue="calendar")
6                                     )
7 print(await client.execute_workflow(CalendarWorkflow.run,
8                                     "Thanks for sending the slides, they look great!",
9                                     id="calendar-2",
10                                    task_queue="calendar")
11                                    )
```

```
added to calendar
no event found in email
```



Making a Durable Workflow

tinycal

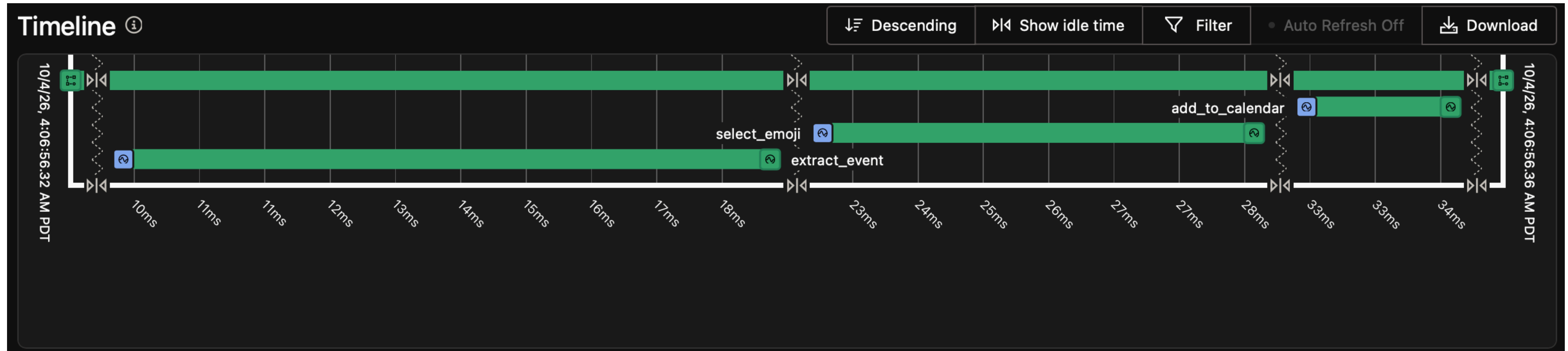
<Today>Oct 4 – Oct 10, 2026

liveClear

Sun4 no events	Mon5 1:30pm – 2:50pm CS329Z Lecture x	Tue6 no events	Wed7 11:00am – 12:00pm Dentist Appointment x 1:30pm – 2:50pm CS329Z Lecture x 3:30pm – 4:00pm Coffee with John x	Thu8 3:00pm – 4:00pm Office hours x	Fri9 10:00am – 11:00am 17 meeting x	Sat10 no events
--------------------------------------	---	--------------------------------------	---	--	--	---------------------------------------



Making a Durable Workflow



Retries



Retrying Flaky Activities

```
1  # This activity simulates a flaky service that fails twice before succeeding.
2
3  @activity.defn(name="select_emoji") # activities are matched by name
4  def flaky_select_emoji(title: str) -> str:
5      print("select_emoji attempt", activity.info().attempt)
6      if activity.info().attempt < 3:
7          raise RuntimeError("503 Service Unavailable")
8      return emoji_cot(event_name=title).emoji
```

✓ 0.0s



Retrying Flaky Activities

```
1  # Activities fail: rate limits, timeouts, flaky networks.
2  #       The server retries the activity for you, according to the `RetryPolicy`.
3
4  worker = Worker(client, task_queue="retries",
5                 workflows=[CalendarWorkflow],
6                 activities=[extract_event, flaky_select_emoji, add_to_calendar],
7                 activity_executor=ThreadPoolExecutor(4))
8  retry_worker_task = asyncio.create_task(worker.run())
9
10 await client.execute_workflow(CalendarWorkflow.run,
11                               "Go for a walk on Tuesday, October 6, 2026 at 9am," \
12                               " should take 30 minutes.", id="retry-1", task_queue="retries")
```

✓ 4.1s

```
select_emoji attempt 1
select_emoji attempt 2
select_emoji attempt 3
```

```
'added to calendar'
```



Retrying Flaky Activities

tinykal

< Today > Oct 4 – Oct 10, 2026

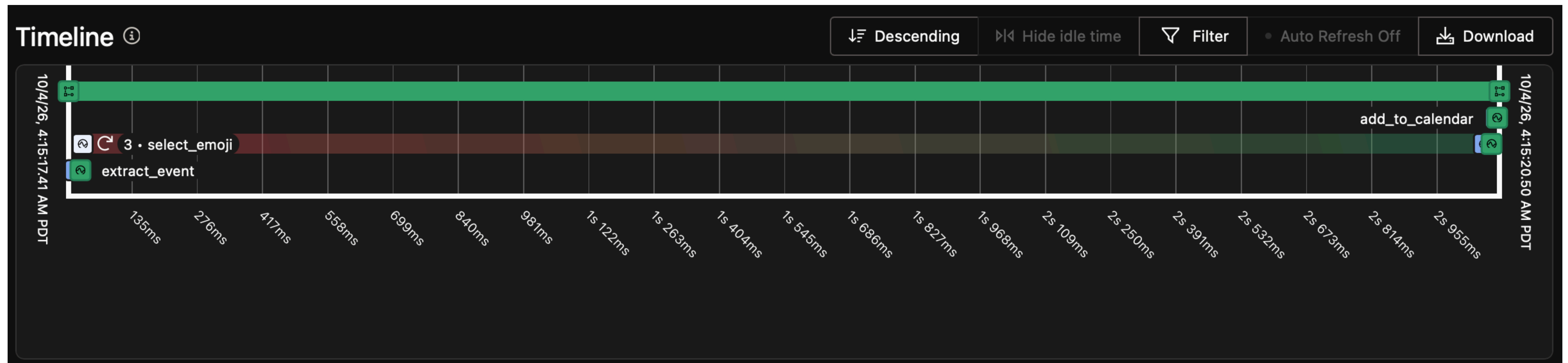
live

Clear

Sun 4 no events	Mon 5 1:30pm – 2:50pm CS329Z Lecture x	Tue 6 9:00am – 9:30am Go for a walk x	Wed 7 11:00am – 12:00pm Dentist Appointment x 1:30pm – 2:50pm CS329Z Lecture x 3:30pm – 4:00pm Coffee with John After lectures x	Thu 8 3:00pm – 4:00pm Office hours x	Fri 9 10:00am – 11:00am meeting x	Sat 10 no events
---	--	---	--	--	---	--



Retrying Flaky Activities



Server Based Orchestration



Server Based Timing

```
1  # Add the event now, then sleep until it's time for a reminder.
2  #      A workflow can sleep for a months or even years in one line.
3  @workflow.defn(sandboxed=False)
4  class ReminderWorkflow:
5      @workflow.run
6      async def run(self, email: str, seconds: int) -> str:
7          status = await workflow.execute_child_workflow(CalendarWorkflow.run,
8              email, id="reminder-1-calendar")
9          await workflow.sleep(seconds)
10         return f"Reminder! ({email})"
11
12  worker = Worker(client, task_queue="reminders",
13                 workflows=[ReminderWorkflow, CalendarWorkflow],
14                 activities=[extract_event, select_emoji, add_to_calendar],
15                 activity_executor=ThreadPoolExecutor(4))
16  reminder_worker_task = asyncio.create_task(worker.run())
```

✓ 0.0s



Server Based Timing

```
1 # Start it, let the calendar steps finish, then kill the worker in the middle of the sleep.
2 handle = await client.start_workflow(ReminderWorkflow.run,
3     args=["Lab meeting on Thursday, October 8th, 2026 at 11am for 60 minutes.", 20],
4     id="reminder-1",
5     task_queue="reminders"
6 )
7 await asyncio.sleep(10)
8 reminder_worker_task.cancel()
9 print("workflow status:", (await handle.describe()).status.name)      # still RUNNING, with no worker alive
```

✓ 10.0s

workflow status: RUNNING



Server Based Timing

```
1  # Bring a worker back. The timer fires and the workflow finishes.
2  #      The LLM steps are not re-run: their results were already recorded.
3  worker = Worker(client, task_queue="reminders",
4                  workflows=[ReminderWorkflow, CalendarWorkflow],
5                  activities=[extract_event, select_emoji, add_to_calendar],
6                  activity_executor=ThreadPoolExecutor(4)
7                  )
8  reminder_worker_task = asyncio.create_task(worker.run())
9
10 print(await handle.result())
11 print([e["title"] for e in tinycal.list_events("2026-10-08")])      # one Lab meeting, not two
```

✓ 10.0s

```
Reminder! (Lab meeting on Thursday, October 8th, 2026 at 11am for 60 minutes.)
['Lab meeting', 'Office hours']
```



Server Based Timing

tinykal

<

Today

>

Oct 4 – Oct 10, 2026

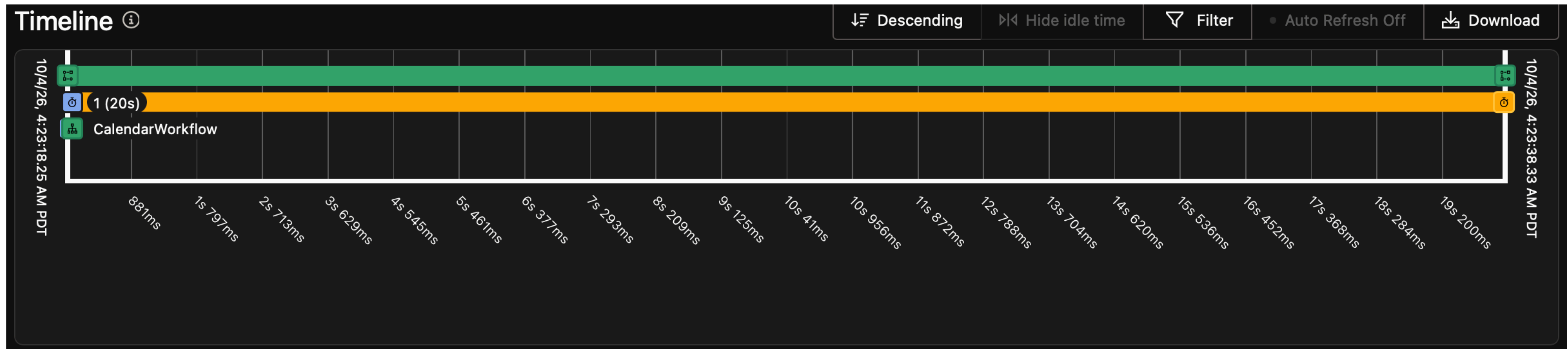
live

Clear

Sun4 no events	Mon5 1:30pm – 2:50pm CS329Z Lecture	Tue6 9:00am – 9:30am Go for a walk	Wed7 11:00am – 12:00pm Dentist Appointment 1:30pm – 2:50pm CS329Z Lecture 3:30pm – 4:00pm Coffee with John	Thu8 11:00am – 12:00pm Lab meeting 3:00pm – 4:00pm Office hours	Fri9 10:00am – 11:00am meeting	Sat10 no events
--------------------------------------	--	---	---	--	---	---------------------------------------



Server Based Timing



Additional Observability Frameworks



MLFlow

The screenshot displays the MLflow web interface in a browser. The browser's address bar shows 'MLflow' and a search bar with the text 'Search Google or type a URL'. The MLflow interface has a dark sidebar on the left with the 'mlflow 2.18.0' logo and a section titled 'Experiments'. Under 'Experiments', there is a search bar and two entries: 'Default' (unchecked) and 'dspy-paper-sur' (checked). The main content area is titled 'LM.__call__' and has a search bar. Below the search bar, there is a table with the following data:

Task name
LM.__call__ 0.01s

At the bottom of the table, there is a button labeled 'Show timeline'. To the right of the table, there are tabs for 'Inputs / Outputs', 'Attributes', and 'Events'. The 'Inputs / Outputs' tab is selected. Under 'Inputs', there are two sections: 'prompt' and 'messages'. The 'prompt' section has a 'Markdown' dropdown and a copy icon. The 'messages' section has a copy icon. The 'Outputs' section is also visible, showing a list of outputs with a 'See more' link.



LangSmith

ProductionTraces

Project

TOTAL RUNS

578

TOTAL TOKENS

23,010

LATENCY

P99 1.20s

P50 3.20s

FEEDBACK

helpfulness: 1.00

coherence: 1.00

relevance: 1.00

Runs

Setup

☐	Name	Input	Latency	Feedback
☐	LLMChain	taylor swift annual tick...	3.20s	coherence 1.00
☐	LLMChain	superbowl points 2022	1.20s	conciseness 1.00
☐	LLMChain	favorite parrot phrase	2.50s	relevance 1.00
☐	LLMChain	best neighbourhood in SF..	7.60s	relevance 1.00

Filters

Full-Text Search

Search...

Feedback

☒ Conciseness

24

☒ Correctness==1

24

☒ Relevance

1

☒ Harmfulness==0

24

Feedback Source

☐ App

24

Name

☐ LLM Chain

156

Run Type

☐ Chain

99

Status

☐ Success

155

☐ Pending

1

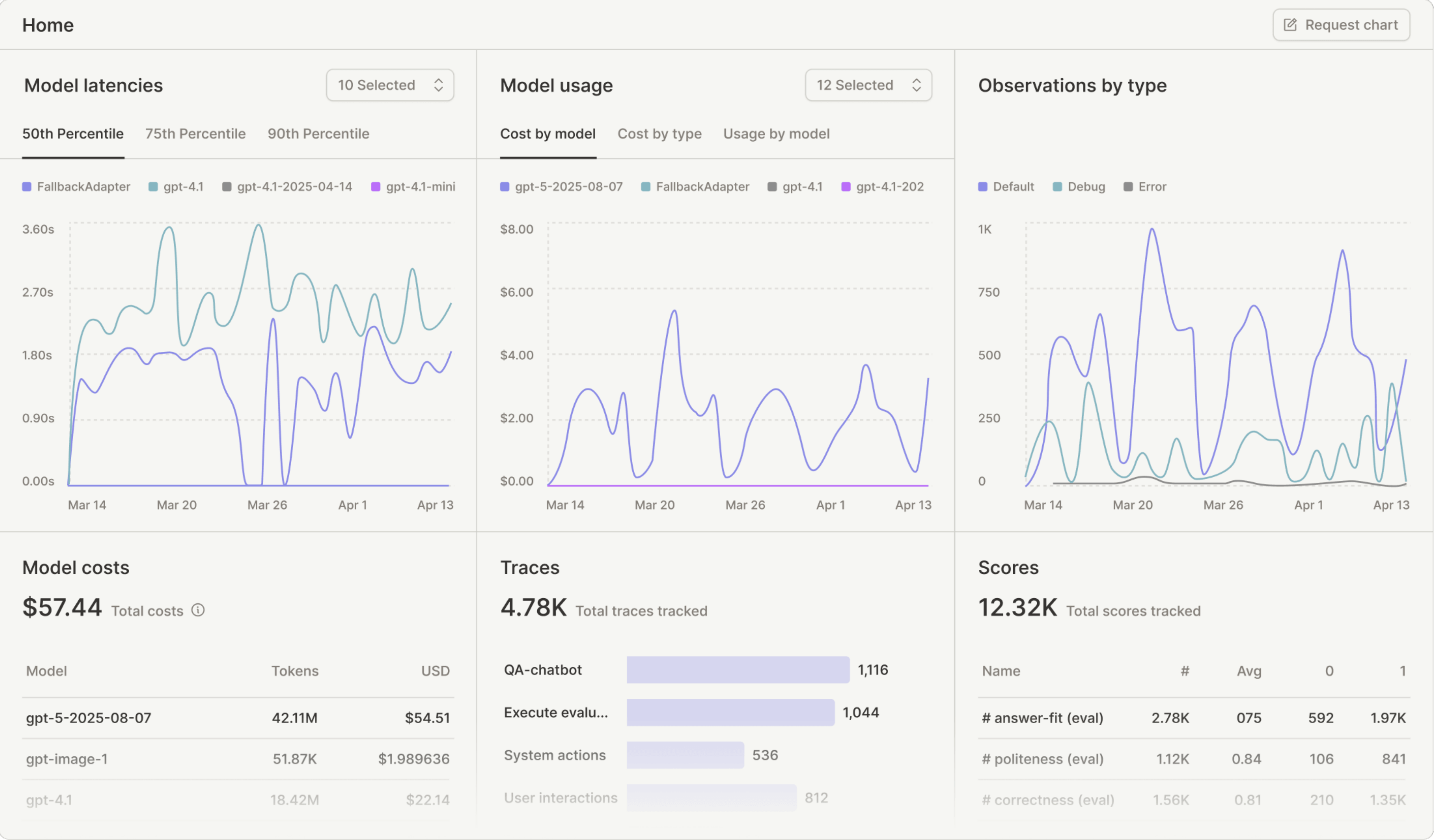
Other

☐ Latency >= 10s

☐ Tokens >= 1,000



LangFuse



Recap

- **Orchestration keeps agents reliable**
- **Observability enables debugging and monitoring**
- Pulling best practices and tools from software engineering will carry over into agent engineering as well.
- Setting up evals, monitoring, durable execution, etc. is what separates building agents from engineering agents.



Best Practices and Challenges

“Engineer” Agents?

Workflows

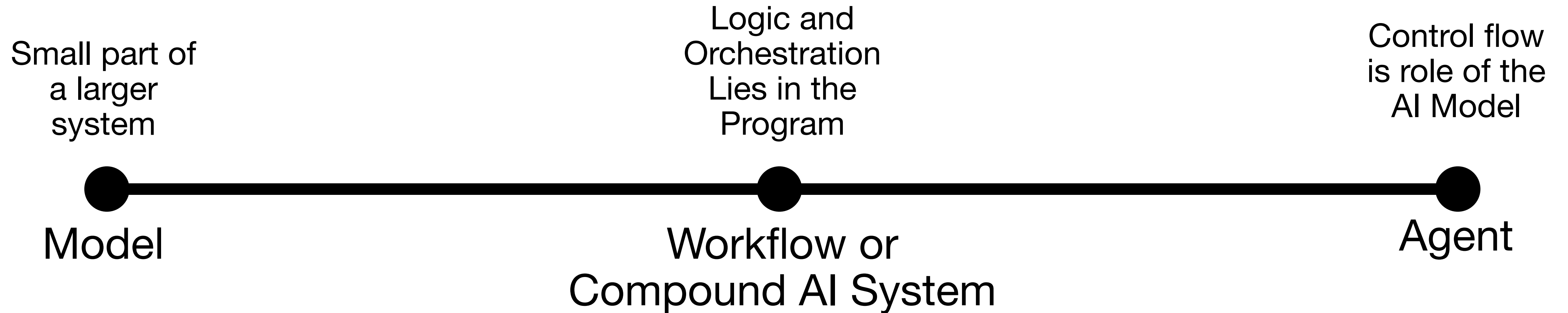
Agent Harnesses

Orchestration

Best Practices



Keep it Simple

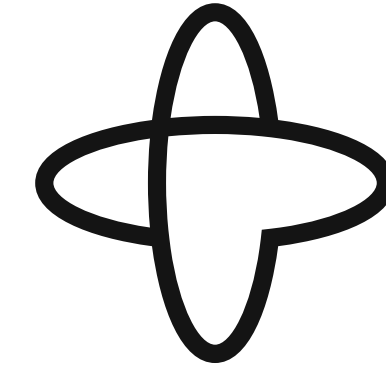


- You might not always need to build an agent. Sometimes a workflow is right for the task.
- When you do build an agent think carefully about what tools you should include.
- If you have a lot of data you may even be able to optimize a model end to end.



Make Failures Observable and Recoverable

- Trace model calls, tool calls, state changes, and errors.
- Set timeouts, retry limits, and execution budgets.
- Log outputs and diagnose failures.
- Monitor the state of your system.



Temporal

mlflow

 Langfuse

...

```
● Bash(sleep 30; grep -E "STATS|MASKED|CHAIN|Traceback|Error" /private/tmp/claude-501/-Users-michaelryan-Documents-School-Stanf  
esearch-marin/64e541da-73a7-421b-83...)
  └─ Error: Blocked: sleep 30 followed by: grep -E "STATS|MASKED|CHAIN|Traceback|Error"  
    /private/tmp/claude-501/-Users-michaelryan-Documents-School-Stanford-Research-marin/64e541da-73a7-421b-831c-6bb5be86cfc7/s  
hpad/chain2.log. To wait for a condition, use  
    Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run_in_backgro  
true. Do not chain shorter sleeps to work around  
    this block.
```

You can raise the error to the agent to try again.



Example: Shepherd

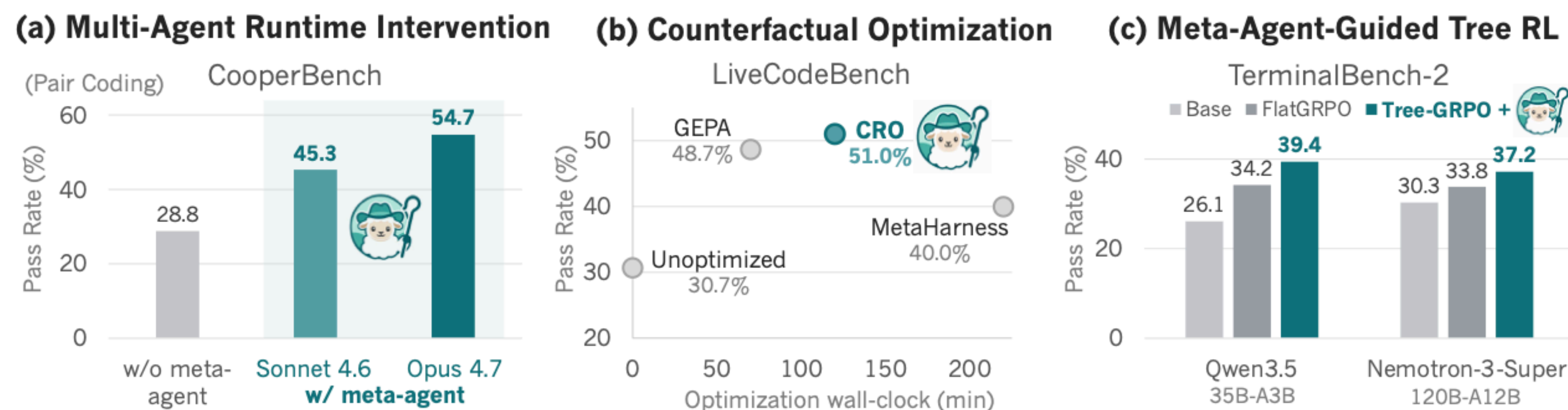
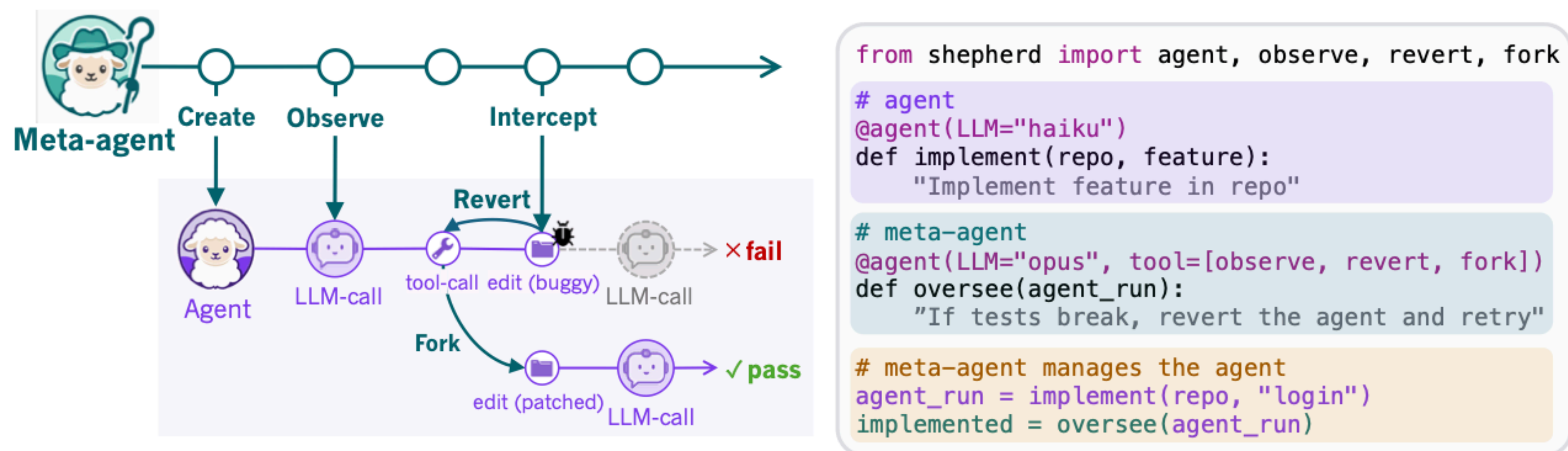


Figure 1: **SHEPHERD** meta-agents. *Top*: A meta-agent optimizes an agent's execution trace. *Bottom*: Results from three meta-agents: (A) runtime intervention; (B) meta-optimization; (C) Tree-GRPO.



Run Evaluations

- The best way to make design decisions is to ground them in evals.
- Evaluations enable optimization
- Building a proper eval is one of the most important steps when making an AI system.
- Choose your metrics wisely (accuracy, cost, latency, etc.)
- Test representative tasks, edge cases, and failure scenarios.



Run Evaluations

TERMINAL-BENCH



SWE-bench

DeepSWE

Your Custom
Domain Eval!

... Unless you are building a general coding agent harness or testing a general agent system improvement, the standard coding benchmarks are NOT what you should climb on.



Challenges / Limitations

- Abstractions lift you further from the underlying system details, so ensure you **know what is going on under the hood.**
- When building a complex AI system it can be challenging to identify where the failures come from. **Make sure to look at the traces!**
- Building on top of frameworks adds dependencies and reliance on external libraries to be maintained.



Recap

- Engineer Agents by **Building -> Running -> Evaluating -> Improving**. The engineering process is a cycle!
- Start simple and justify added complexity.
- Design for failure and recovery.
- Use evidence and evaluation to assess quality, cost, and latency.
- Fully understand how your system works.



Summary

“Engineer” Agents?

Workflows

Agent Harnesses

Orchestration

Best Practices



Summary

- ✓ What does it mean to “Engineer” Agents?
- ✓ Frameworks for Workflows and Programs (DSPy, LangGraph)
- ✓ Open Source Agent Harnesses
- ✓ Orchestration and Telemetry (Temporal)
- ✓ Best Practices and Challenges

